

Computer Networks (2025-2026)

Part 3: Transport Layer

Jeroen Famaey

Andrei Belogaev

{jeroen.famaey, andrei.belogaev}@uantwerpen.be

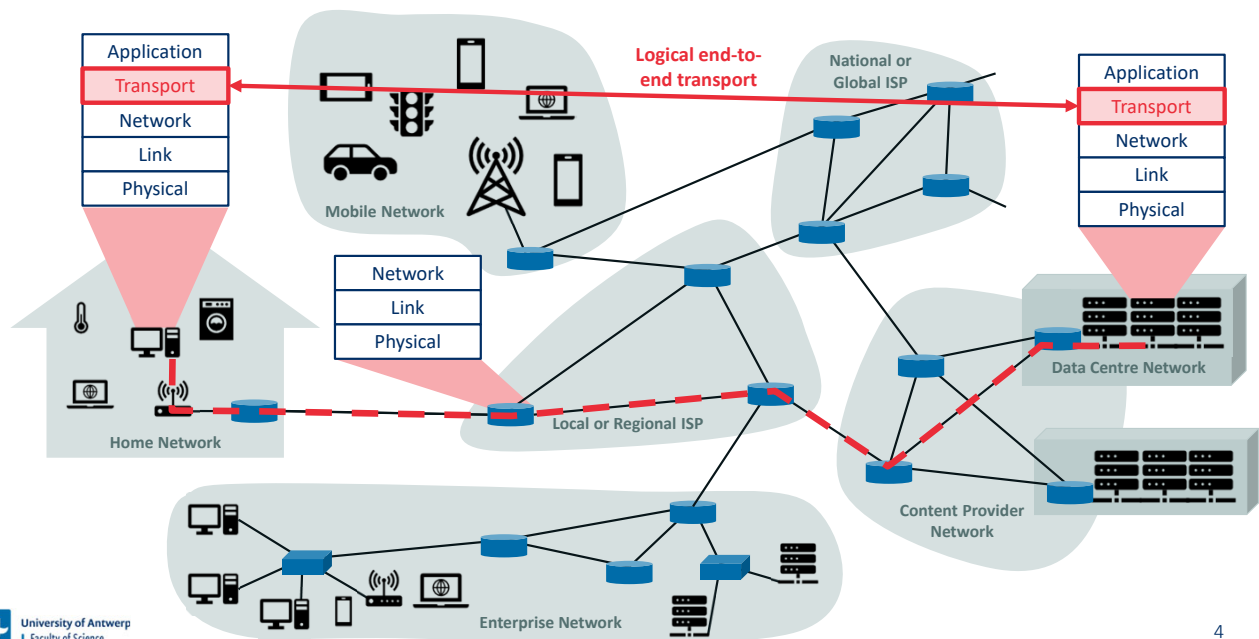
Table of contents

1. Transport-layer services
2. User Datagram Protocol (UDP)
3. Reliable data transfer
4. Transport Control Protocol (TCP)
5. TCP congestion control

Residing between the application and network layers, the transport layer is a central piece of the layered network architecture. It has the critical role of providing communication services directly to the application processes running on different hosts. We'll begin by discussing the relationship between the transport and network layers. This sets the stage for examining the first critical function of the transport layer—extending the network layer's delivery service between two end systems to a delivery service between two application-layer processes running on the end systems. We'll then return to principles and confront one of the most fundamental problems in computer networking—how two entities can communicate reliably over a medium that may lose and corrupt data. We'll next move on to a second fundamentally important problem in networking—controlling the transmission rate of transport-layer entities in order to avoid, or recover from, congestion within the network. We'll consider the causes and consequences of congestion, as well as commonly used congestion-control techniques. After obtaining a solid understanding of the issues behind congestion control, we'll study TCP's approach to congestion control.

Transport-layer services

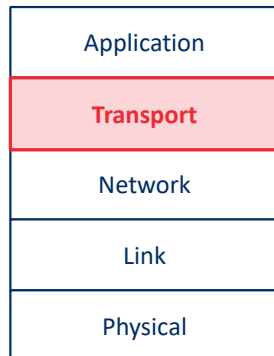
Transport layer provides a logical connection between applications



A transport-layer protocol provides for **logical communication** between application processes running on different hosts. By *logical communication*, we mean that from an application's perspective, it is as if the hosts running the processes were directly connected; in reality, the hosts may be on opposite sides of the planet, connected via numerous routers and a wide range of link types. Application processes use the logical communication provided by the transport layer to send messages to each other, free from the worry of the details of the physical infrastructure used to carry these messages.

Transport-layer protocols are implemented in the end systems but not in network routers. On the sending side, the transport layer converts the application-layer messages it receives from a sending application process into transport-layer packets, known as transport-layer **segments** in Internet terminology. This is done by (possibly) breaking the application messages into smaller chunks and adding a transport-layer header to each chunk to create the transport-layer segment. The transport layer then passes the segment to the network layer at the sending end system, where the segment is encapsulated within a network-layer packet (a datagram) and sent to the destination. It's important to note that network routers act only on the network-layer fields of the datagram; that is, they do not examine the fields of the transport-layer segment encapsulated with the datagram. On the receiving side, the network layer extracts the transport-layer segment from the datagram and passes the segment up to the transport layer. The transport layer then processes the received segment, making the data in the segment available to the receiving application. Recall that the transport layer lies just above the network layer in the protocol stack. Whereas a transport-layer protocol provides logical communication between *processes*, a network-layer protocol provides logical-communication between *hosts*.

The Internet's transport Layer services



- Transports application-layer message between end-systems
- Transport-layer packets are referred to as **segments**
- **General services**
 - (De)multiplexing: Deliver segments to the correct process running on the end host
 - Error checking: Check the integrity of received segments using error-detection header field
- **Transport Control Protocol (TCP)**
 - Connection-oriented
 - Reliable (retransmits lost packets)
 - Flow control (sender/receiver speed matching)
 - Congestion control (throttles transmission rate to avoid network congestion)
- **User Datagram Protocol (UDP)**
 - Connection-less
 - Unreliable, no flow control, no congestion control

Before proceeding with our brief introduction of UDP and TCP, it will be useful to say a few words about the Internet's network layer. The Internet's network-layer protocol has a name—IP, for Internet Protocol. IP provides logical communication between hosts. The IP service model is a **best-effort delivery service**. This means that IP makes its “best effort” to deliver segments between communicating hosts, *but it makes no guarantees*. In particular, it does not guarantee segment delivery, it does not guarantee orderly delivery of segments, and it does not guarantee the integrity of the data in the segments. For these reasons, IP is said to be an **unreliable service**. We also mention here that every host has at least one network-layer address, a so-called IP address.

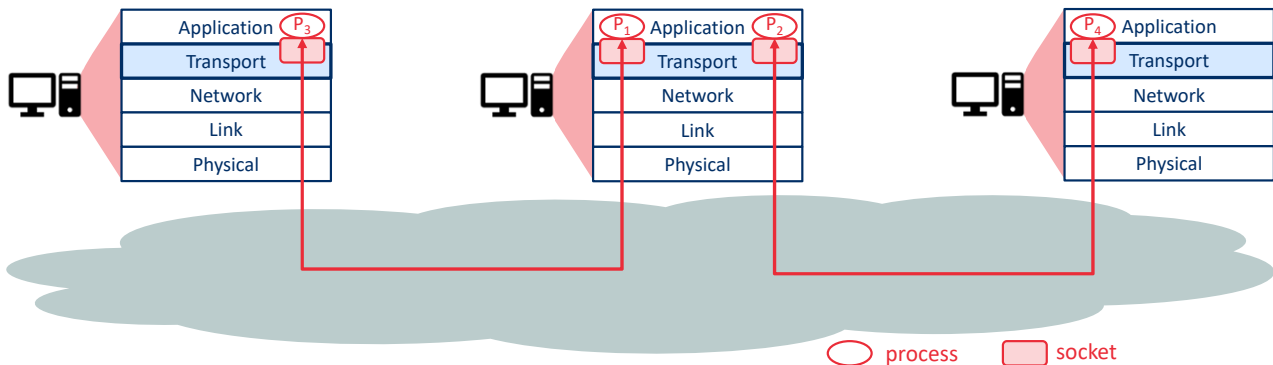
Recall that the Internet makes two distinct transport-layer protocols available to the application layer. One of these protocols is **UDP** (User Datagram Protocol), which provides an unreliable, connectionless service to the invoking application. The second of these protocols is **TCP** (Transmission Control Protocol), which provides a reliable, connection-oriented service to the invoking application. When designing a network application, the application developer must specify one of these two transport protocols.

The most fundamental responsibility of UDP and TCP is to extend IP's delivery service between two end systems to a delivery service between two processes running on the end systems. Extending host-to-host delivery to process-to-process delivery is called **transport-layer multiplexing** and **demultiplexing**. UDP and TCP also provide integrity checking by including error-detection fields in their segments' headers. These two minimal transport-layer services—process-to-process data delivery and error checking—are the only two services that UDP provides! In particular, like IP, UDP is an unreliable service—it does not

guarantee that data sent by one process will arrive intact (or at all!) to the destination process.

TCP, on the other hand, offers several additional services to applications. First and foremost, it provides **reliable data transfer**. Using flow control, sequence numbers, acknowledgments, and timers, TCP ensures that data is delivered from sending process to receiving process, correctly and in order. TCP thus converts IP's unreliable service between end systems into a reliable data transport service between processes. TCP also provides **congestion control**. Congestion control is not so much a service provided to the invoking application as it is a service for the Internet as a whole, a service for the general good. Loosely speaking, TCP congestion control prevents any one TCP connection from swamping the links and routers between communicating hosts with an excessive amount of traffic. TCP strives to give each connection traversing a congested link an equal share of the link bandwidth. This is done by regulating the rate at which the sending sides of TCP connections can send traffic into the network. UDP traffic, on the other hand, is unregulated. An application using UDP transport can send at any rate it pleases, for as long as it pleases.

Multiplexing and demultiplexing



- **Multiplexing:** Encapsulating data from different sockets into segments and passing them to the network layer
- **Demultiplexing:** Identifying the receiving socket based on header fields and directing the data to that socket
- Identification of the correct socket (and thus process) is done based on a **source and destination port number**

At the destination host, the transport layer receives segments from the network layer just below. The transport layer has the responsibility of delivering the data in these segments to the appropriate application process running in the host. Let's take a look at an example. Suppose you are sitting in front of your computer, and you are downloading Web pages while running one FTP session and two Telnet sessions. You therefore have four network application processes running—two Telnet processes, one FTP process, and one HTTP process. When the transport layer in your computer receives data from the network layer below, it needs to direct the received data to one of these four processes. Let's now examine how this is done.

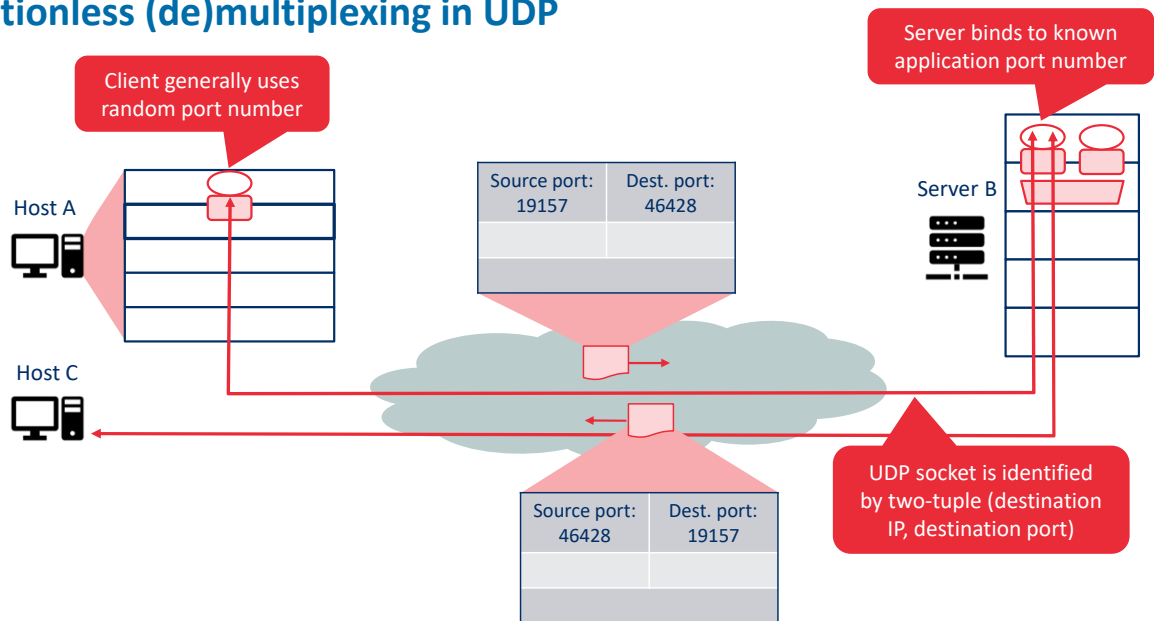
First recall that a process (as part of a network application) can have one or more **sockets**, doors through which data passes from the network to the process and through which data passes from the process to the network. Thus, the transport layer in the receiving host does not actually deliver data directly to a process, but instead to an intermediary socket. Because at any given time there can be more than one socket in the receiving host, each socket has a unique identifier.

Now let's consider how a receiving host directs an incoming transport-layer segment to the appropriate socket. Each transport-layer segment has a set of fields in the segment for this purpose. At the receiving end, the transport layer examines these fields to identify the receiving socket and then directs the segment to that socket. This job of delivering the data in a transport-layer segment to the correct socket is called **demultiplexing**. The job of gathering data chunks at the source host from different sockets, encapsulating each data chunk with header information (that will later be used in demultiplexing) to create segments, and

passing the segments to the network layer is called **multiplexing**.

From the discussion above, we know that transport-layer multiplexing requires (1) that sockets have unique identifiers, and (2) that each segment have special fields that indicate the socket to which the segment is to be delivered. These special fields, are the **source port number field** and the **destination port number field**. Each port number is a 16-bit number, ranging from 0 to 65535. The port numbers ranging from 0 to 1023 are called **well-known port numbers** and are restricted, which means that they are reserved for use by well-known application protocols such as HTTP (which uses port number 80) and FTP (which uses port number 21). The list of well-known port numbers is given in RFC 1700 and is updated at <http://www.iana.org> (RFC 3232). When we develop a new application, we must assign the application a port number.

Connectionless (de)multiplexing in UDP



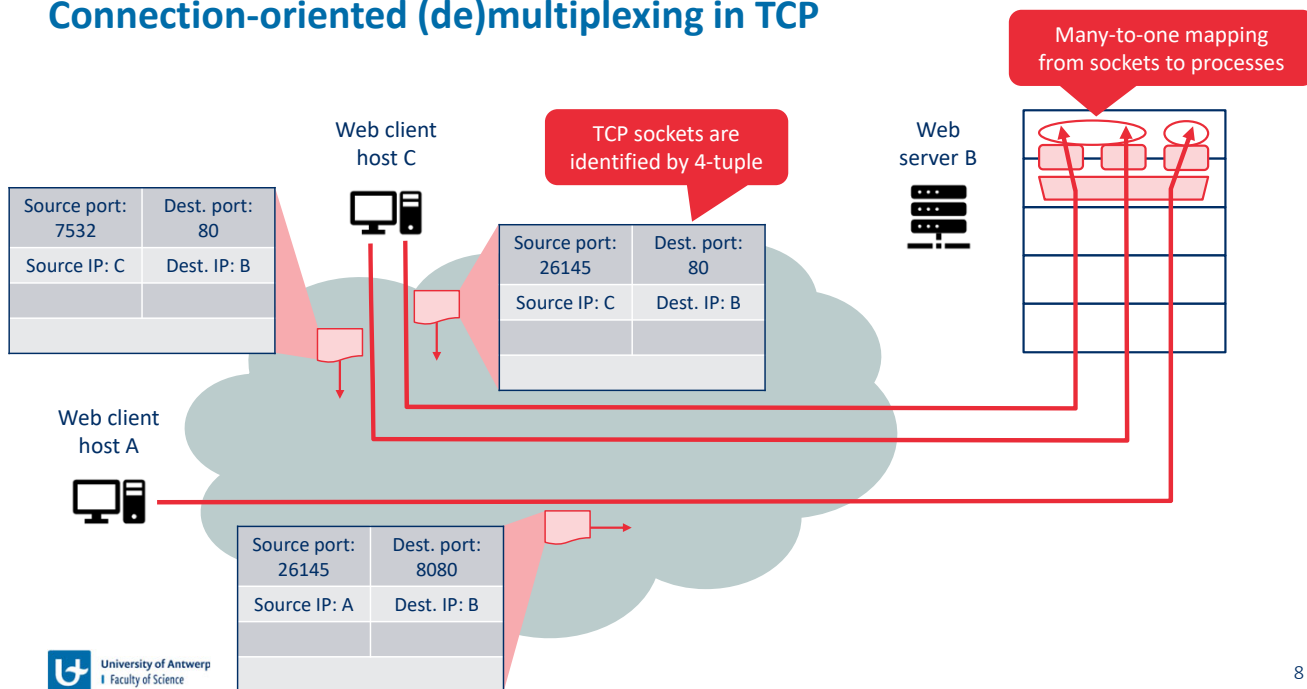
When creating a UDP socket, we can either let the transport layer assign a random port number (in the range 1024 to 65535) or bind it to a specific port number. If the application developer writing the code were implementing the server side of a “well-known protocol,” then the developer would have to assign the corresponding well-known port number. Typically, the client side of the application lets the transport layer automatically (and transparently) assign the port number, whereas the server side of the application assigns a specific port number.

With port numbers assigned to UDP sockets, we can now precisely describe UDP multiplexing/demultiplexing. Suppose a process in Host A, with UDP port 19157, wants to send a chunk of application data to a process with UDP port 46428 in Host B. The transport layer in Host A creates a transport-layer segment that includes the application data, the source port number (19157), the destination port number (46428), and two other values (which will be discussed later, but are unimportant for the current discussion). The transport layer then passes the resulting segment to the network layer. The network layer encapsulates the segment in an IP datagram and makes a best-effort attempt to deliver the segment to the receiving host. If the segment arrives at the receiving Host B, the transport layer at the receiving host examines the destination port number in the segment (46428) and delivers the segment to its socket identified by port 46428. Note that Host B could be running multiple processes, each with its own UDP socket and associated port number. As UDP segments arrive from the network, Host B directs (demultiplexes) each segment to the appropriate socket by examining the segment’s destination port number.

It is important to note that a **UDP socket is fully identified by a two-tuple consisting of a**

destination IP address and a destination port number. As a consequence, if two UDP segments have different source IP addresses and/or source port numbers, but have the same *destination* IP address and *destination* port number, then the two segments will be directed to the same destination process via the same destination socket. You may be wondering now, what is the purpose of the source port number? As shown in the figure, in the A-to-B segment the source port number serves as part of a “return address”—when B wants to send a segment back to A, the destination port in the B-to-A segment will take its value from the source port value of the A-to-B segment. The complete return address is A’s IP address and the source port number.

Connection-oriented (de)multiplexing in TCP



In order to understand TCP demultiplexing, we have to take a close look at TCP sockets and TCP connection establishment. One subtle difference between a TCP socket and a UDP socket is that a TCP socket is identified by a four-tuple: (source IP address, source port number, destination IP address, destination port number). Thus, when a TCP segment arrives from the network to a host, the host uses all four values to direct (demultiplex) the segment to the appropriate socket. In particular, and in contrast with UDP, two arriving TCP segments with different source IP addresses or source port numbers will (with the exception of a TCP segment carrying the original connection-establishment request) be directed to two different sockets.

The TCP server application has a “welcoming socket,” that waits for connection-establishment requests from TCP clients on a specific port (e.g., port 80 for HTTP). The TCP client then creates a socket and sends a connection establishment request to the server’s IP address and chosen server port. A connection-establishment request is nothing more than a TCP segment with destination port number and a special connection-establishment bit set in the TCP header. The segment also includes a source port number that was chosen by the client. When the host operating system of the computer running the server process receives the incoming connection-request segment with the specific destination port, it locates the server process that is waiting to accept a connection on that port number. The transport layer at the server notes the following four values in the connection-request segment: (1) the source port number in the segment, (2) the IP address of the source host, (3) the destination port number in the segment, and (4) its own IP address. The newly created connection socket is identified by these four values; all subsequently arriving segments whose source port, source IP address, destination port, and destination IP address match these four values will

be demultiplexed to this socket. With the TCP connection now in place, the client and server can now send data to each other.

The server host may support many simultaneous TCP connection sockets, with each socket attached to a process, and with each socket identified by its own four-tuple. When a **TCP segment arrives at the host, all four fields (source IP address, source port, destination IP address, destination port) are used** to direct (demultiplex) the segment to the appropriate socket. The situation is illustrated in the figure, in which Host C initiates two HTTP sessions to server B, and Host A initiates one HTTP session to B. Hosts A and C and server B each have their own unique IP address—A, C, and B, respectively. Host C assigns two different source port numbers (26145 and 7532) to its two HTTP connections. Because Host A is choosing source port numbers independently of C, it might also assign a source port of 26145 to its HTTP connection. But this is not a problem—server B will still be able to correctly demultiplex the two connections having the same source port number, since the two connections have different source IP addresses.

We mention that there is not always a one-to-one correspondence between connection sockets and processes. In fact, today's high-performing Web servers often use only one process, and create a new thread with a new connection socket for each new client connection. A thread can be viewed as a lightweight subprocess.

User Datagram Protocol (UDP)

User Datagram Protocol (UDP) – RFC 768

- UDP is “bare bones”: connection-less, unreliable, no flow or congestion control, ...
- Then why use UDP instead of TCP?
 - Finer application-level control over what data is sent, and when
 - No connection establishment (i.e., less delay before data can be sent)
 - No connection state (i.e., less overhead per connection on the server)
 - Smaller packet header overhead (i.e., 8 bytes for UDP vs. 20 bytes for TCP)

UDP, defined in [RFC 768], does just about as little as a transport protocol can do. Aside from the multiplexing/demultiplexing function and some light error checking, it adds nothing to IP. In fact, if the application developer chooses UDP instead of TCP, then the application is almost directly talking with IP. UDP takes messages from the application process, attaches source and destination port number fields for the multiplexing/ demultiplexing service, adds two other small fields, and passes the resulting segment to the network layer. The network layer encapsulates the transport-layer segment into an IP datagram and then makes a best-effort attempt to deliver the segment to the receiving host. If the segment arrives at the receiving host, UDP uses the destination port number to deliver the segment's data to the correct application process. Note that with UDP there is no handshaking between sending and receiving transport-layer entities before sending a segment. For this reason, UDP is said to be *connectionless*.

Now you might be wondering why an application developer would ever choose to build an application over UDP rather than over TCP. Isn't TCP always preferable, since TCP provides a reliable data transfer service, while UDP does not? The answer is no, as some applications are better suited for UDP for the following reasons:

- ***Finer application-level control over what data is sent, and when.*** Under UDP, as soon as an application process passes data to UDP, UDP will package the data inside a UDP segment and immediately pass the segment to the network layer. TCP, on the other hand, has a congestion-control mechanism that throttles the transport-layer TCP sender when one or more links between the source and destination hosts become excessively congested. TCP will also continue to resend a segment until the receipt of the segment has been acknowledged by the destination, regardless of how long reliable delivery

takes. Since real-time applications often require a minimum sending rate, do not want to overly delay segment transmission, and can tolerate some data loss, TCP's service model is not particularly well matched to these applications' needs. These applications can use UDP and implement, as part of the application, any additional functionality that is needed beyond UDP's no-frills segment-delivery service.

- **No connection establishment.** TCP uses a three-way handshake before it starts to transfer data. UDP just blasts away without any formal preliminaries. Thus UDP does not introduce any delay to establish a connection. This is probably the principal reason why DNS runs over UDP rather than TCP—DNS would be much slower if it ran over TCP. HTTP uses TCP rather than UDP, since reliability is critical for Web pages with text. But, as we briefly discussed in Section 2.2, the TCP connection-establishment delay in HTTP is an important contributor to the delays associated with downloading Web documents. Indeed, the QUIC protocol (Quick UDP Internet Connection), used in Google's Chrome browser, uses UDP as its underlying transport protocol and implements reliability in an application-layer protocol on top of UDP.
- **No connection state.** TCP maintains connection state in the end systems. This connection state includes receive and send buffers, congestion-control parameters, and sequence and acknowledgment number parameters. This state information is needed to implement TCP's reliable data transfer service and to provide congestion control. UDP, on the other hand, does not maintain connection state and does not track any of these parameters. For this reason, a server devoted to a particular application can typically support many more active clients when the application runs over UDP rather than TCP.
- **Small packet header overhead.** The TCP segment has 20 bytes of header overhead in every segment, whereas UDP has only 8 bytes of overhead.

Exercise: Calculate the transport layer overhead for UDP and TCP



Given a header size of 8 bytes for UDP and 20 bytes for TCP, calculate the percentage overhead of the header for an application-layer message of 16 bytes and 1024 bytes.

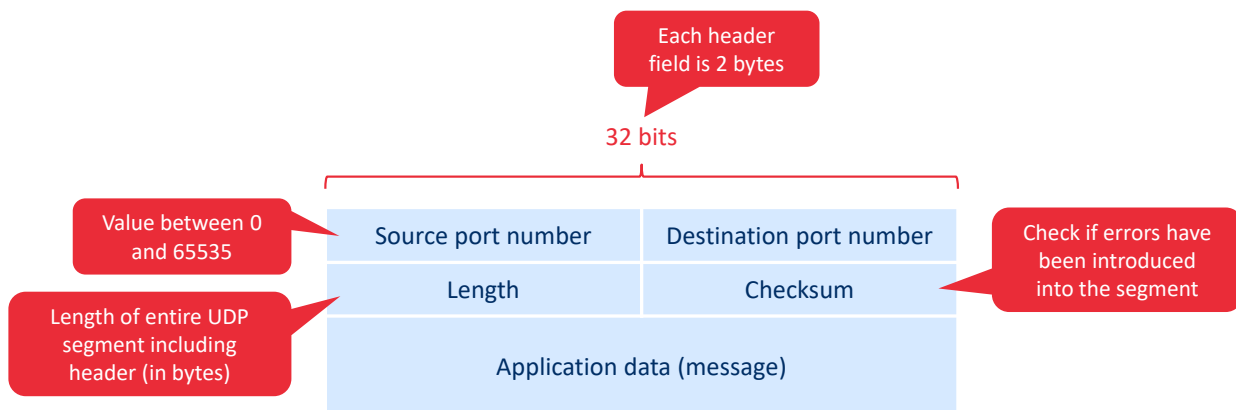
Transport protocol used by popular Internet applications

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Secure remote terminal access	SSH	TCP
Web	HTTP, HTTP/3	TCP (HTTP), UDP (HTTP/3)
File transfer	FTP	TCP
Remote file server	NFS	UDP or TCP (v3+)
Streaming multimedia	DASH	TCP
Internet telephony	Typically proprietary	UDP or TCP
Network management	SNMP	Typically UDP
Name translation	DNS	Typically UDP

The table lists popular Internet applications and the transport protocols that they use. As we expect, e-mail, remote terminal access, and file transfer run over TCP—all these applications need the reliable data transfer service of TCP. Early versions of HTTP ran over TCP but that more recent versions of HTTP run over UDP, providing their own error control and congestion control (among other services) at the application layer. Nevertheless, many important applications run over UDP rather than TCP. For example, UDP is used to carry network management (SNMP) data. UDP is preferred to TCP in this case, since network management applications must often run when the network is in a stressed state—precisely when reliable, congestion-controlled data transfer is difficult to achieve. Also, as we mentioned earlier, DNS runs over UDP, thereby avoiding TCP’s connection-establishment delays.

Both UDP and TCP are sometimes used today with multimedia applications, such as Internet phone, real-time video conferencing, and streaming of stored audio and video. We just mention now that all of these applications can tolerate a small amount of packet loss, so that reliable data transfer is not absolutely critical for the application’s success. Furthermore, real-time applications, like Internet phone and video conferencing, react very poorly to TCP’s congestion control. For these reasons, developers of multimedia applications may choose to run their applications over UDP instead of TCP. When packet loss rates are low, and with some organizations blocking UDP traffic for security reasons, TCP becomes an increasingly attractive protocol for streaming media transport.

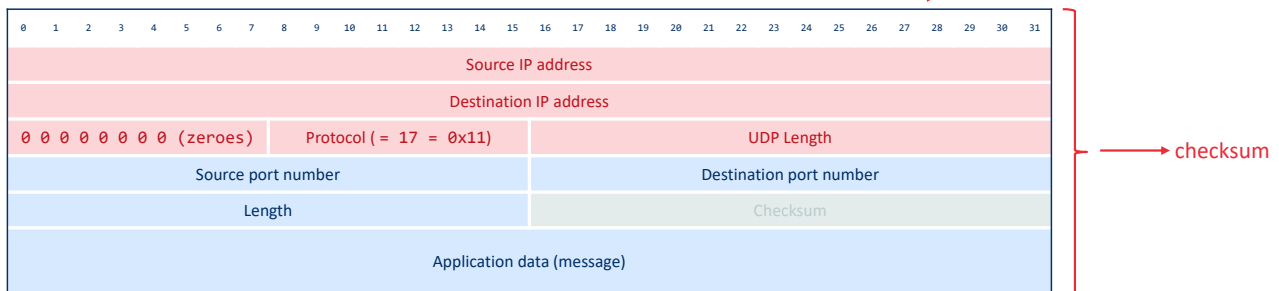
UDP segment structure



The UDP segment structure is defined in RFC 768. The application data occupies the data field of the UDP segment. For example, for DNS, the data field contains either a query message or a response message. For a streaming audio application, audio samples fill the data field. The UDP header has only four fields, each consisting of two bytes. As discussed in the previous section, the port numbers allow the destination host to pass the application data to the correct process running on the destination end system (that is, to perform the demultiplexing function). The length field specifies the number of bytes in the UDP segment (header plus data). An explicit length value is needed since the size of the data field may differ from one UDP segment to the next. The checksum is used by the receiving host to check whether errors have been introduced into the segment. In truth, the checksum is also calculated over a few of the fields in the IP header in addition to the UDP segment. The length field specifies the length of the UDP segment, including the header, in bytes.

UDP checksum

- Sender
 - Calculated by the sender as the 1s complement of the sum of all 16-bit words in the UDP segment (+ IP pseudo header)
 - Overflowing bits are wrapped around
- Receiver:
 - Add all 16-bit words of the segment together with the checksum
 - Result should be 11111111 11111111
 - If an error is detected, the packet is usually discarded



The UDP checksum provides for error detection. That is, the checksum is used to determine whether bits within the UDP segment have been altered (for example, by noise in the links or while stored in a router) as it moved from source to destination. UDP at the sender side performs the 1s complement of the sum of all the 16-bit words in the segment, with any overflow encountered during the sum being wrapped around. This result is put in the checksum field of the UDP segment. The 1s complement is obtained by converting all the 0s to 1s and converting all the 1s to 0s. At the receiver, all four 16-bit words are added, including the checksum. If no errors are introduced into the packet, then clearly the sum at the receiver will be 1111111111111111. If one of the bits is a 0, then we know that errors have been introduced into the packet.

The checksum is calculated on a combination of IP and UDP header fields, called a pseudo header. From the IP header it uses the source IP address, destination IP address, the protocol number (17 for UDP) prepended with 0 zeroes, and the UDP length (total length of the UDP segment, including UDP header fields). It also uses the entire UDP segment (header fields and data), excluding the checksum.

You may wonder why UDP provides a checksum in the first place, as many link-layer protocols (including the popular Ethernet protocol) also provide error checking. The reason is that there is no guarantee that all the links between source and destination provide error checking; that is, one of the links may use a link-layer protocol that does not provide error checking. Furthermore, even if segments are correctly transferred across a link, it's possible that bit errors could be introduced when a segment is stored in a router's memory. Given that neither link-by-link reliability nor in-memory error detection is guaranteed, UDP must

provide error detection at the transport layer, *on an end-end basis*, if the end-end data transfer service is to provide error detection.

Although UDP provides error checking, it does not do anything to recover from an error. Some implementations of UDP simply discard the damaged segment; others pass the damaged segment to the application with a warning.

UDP checksum example

16-bit input words:

```
01100110 01100000
01010101 01010101
10001111 00001100
```

Sum of words:

```
  01100110 01100000
+ 01010101 01010101
-----
  10111011 10110101
+ 10001111 00001100
-----
  01001010 11000010
```

1s complement:

```
~ 01001010 11000010
-----
  10110101 00111101
```

Checksum

Limitations of UDP checksum

- Checksum is optional (an all 0s checksum means it hasn't been calculated)
- All 1-bit errors are detected
- Not all 2-bit errors are caught
- Generally, if the sum of all 16-bit words remains constant, the error will not be detected



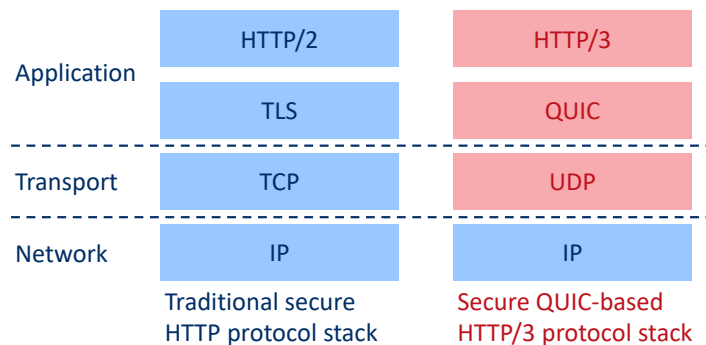
Exercise: Can you think of an example where a 2-bit error is not detected using the UDP checksum?

Original input:

01100110 01100000
01010101 01010101
10001111 00001100

QUIC: A UDP-Based Multiplexed and Secure Transport

- QUIC (originally) stood for Quick UDP Internet Connections (but is now considered the protocol's name)
- Developed and first deployed by Google in 2012
- Standardized since May 2021, as IETF RFC 9000
- About 8,9% of websites use QUIC as of February 2026^[1] (including all Google and Meta webapps)
- Specifically designed to interface with HTTP/3 (IETF RFC 9114, June 2022)



If the transport services needed by an application don't quite fit either the UDP or TCP service models—perhaps an application needs more services than those provided by UDP but does not want all of the particular functionality that comes with TCP or may want different services than those provided by TCP—application designers can always “roll their own” protocol at the application layer. This is the approach taken in the QUIC (Quick UDP Internet Connections) protocol. Specifically, QUIC is a new application-layer protocol designed from the ground up to improve the performance of transport-layer services for secure HTTP. QUIC has already been widely deployed and was standardized in May 2021 as IETF RFC 9000. Google has deployed QUIC on many of its public-facing Web servers, in its mobile video streaming YouTube app, in its Chrome browser, and in Android's Google Search app.

QUIC is an application-layer protocol, using UDP as its underlying transport-layer protocol, and is designed to interface above specifically to a simplified but evolved version of HTTP/2, standardized as HTTP/3. HTTP/3 was standardized by IETF in June 2022, as RFC 9114. The standard describes the mapping of HTTP semantics over QUIC. It also identifies HTTP/2 features that are subsumed by QUIC and describes how HTTP/2 extensions can be ported to HTTP/3.

Major features of QUIC and HTTP/3

- **Connection-oriented and secure**

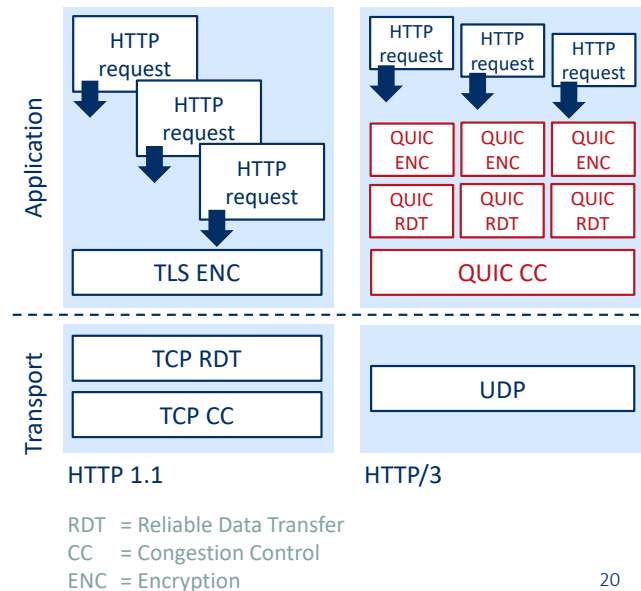
- Single handshake that combines session establishment and authentication/encryption (TCP+TLS separate these)
- Reduces handshake to 1 RTT

- **Streams**

- Multiple application-level streams can be multiplexed over a single QUIC connection
- Quickly add and remove streams
- A stream is a reliable, in-order bi-directional delivery of data between 2 QUIC endpoints (e.g., 1 HTTP/3 object)

- **Reliable, TCP-friendly congestion control**

- Reliability is guaranteed for each QUIC stream separately
- Avoids Head-of-Line (HOL) blocking of multiplexing multiple HTTP responses over a single TCP connection
- Uses TCP NewReno congestion control algorithms



Some of QUIC's major features include:

- **Connection-Oriented and Secure.** Like TCP, QUIC is a connection-oriented protocol between two endpoints. This requires a handshake between endpoints to set up the QUIC connection state. Two pieces of connection state are the source and destination connection ID. All QUIC packets are encrypted, and QUIC combines the handshakes needed to establish connection state with those needed for authentication and encryption, thus providing faster establishment than TCP and HTTP/2, where multiple RTTs are required to first establish a TCP connection, and then establish a TLS connection over the TCP connection.
- **Streams.** QUIC allows several different application-level "streams" to be multiplexed through a single QUIC connection, and once a QUIC connection is established, new streams can be quickly added. A stream is an abstraction for the reliable, in-order bi-directional delivery of data between two QUIC endpoints. In the context of HTTP/3, there would be a different stream for each object in a Web page. Each connection has a connection ID, and each stream within a connection has a stream ID; both of these IDs are contained in a QUIC packet header (along with other header information). Data from multiple streams may be contained within a single QUIC segment, which is carried over UDP.
- **Reliable, TCP-friendly congestion-controlled data transfer.** QUIC provides reliable data transfer to each QUIC stream separately. Since TCP provides reliable, in-order byte delivery, this means that the multiple HTTP requests must be delivered in-order at the destination HTTP server. Thus, if bytes from one HTTP request are lost, the remaining HTTP requests cannot be delivered until those lost bytes are retransmitted and correctly received by TCP at the HTTP server—the so-called HOL blocking problem that we

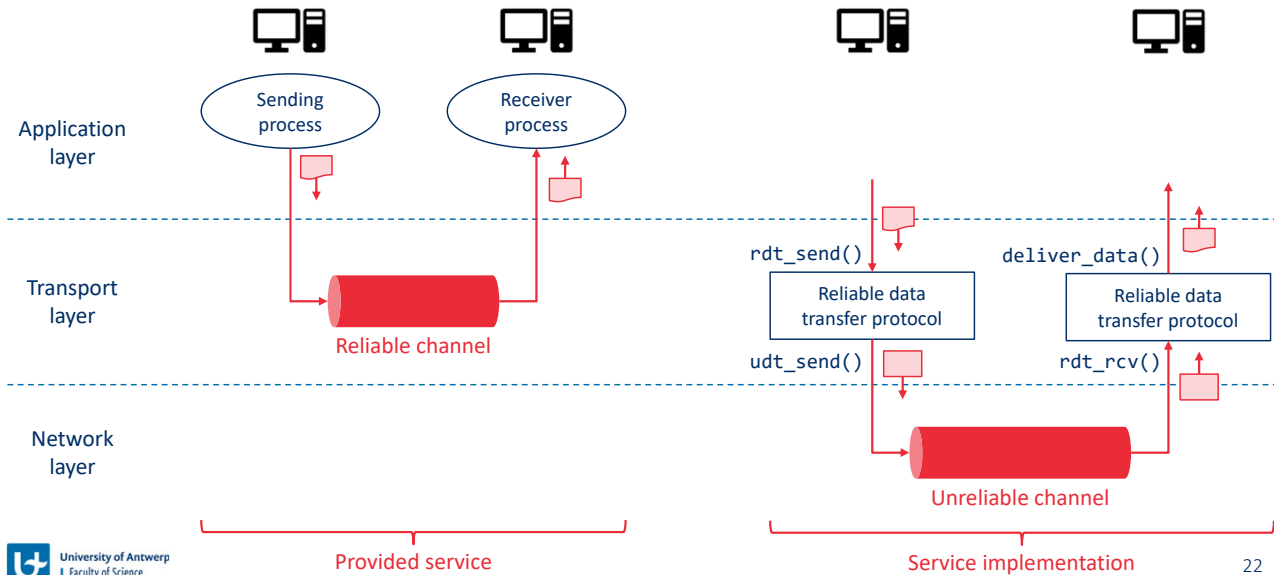
encountered before. Since QUIC provides a reliable in-order delivery on a per-stream basis, a lost UDP segment only impacts those streams whose data was carried in that segment; HTTP messages in other streams can continue to be received and delivered to the application. QUIC provides reliable data transfer using acknowledgment mechanisms similar to TCP's, as specified in [RFC 5681].

QUIC's congestion control is based on TCP NewReno [RFC 6582], a slight modification to the TCP Reno protocol (see later).

Reliable data transfer

We consider the problem of reliable data transfer in a general context. This is appropriate since the problem of implementing reliable data transfer occurs not only at the transport layer, but also at the link layer and the application layer as well. The general problem is thus of central importance to networking. Indeed, if one had to identify a “top-ten” list of fundamentally important problems in all of networking, this would be a candidate to lead the list. In the next section, we’ll examine TCP and show, in particular, that TCP exploits many of the principles that we are about to describe.

Reliable data transfer over an unreliable channel



22

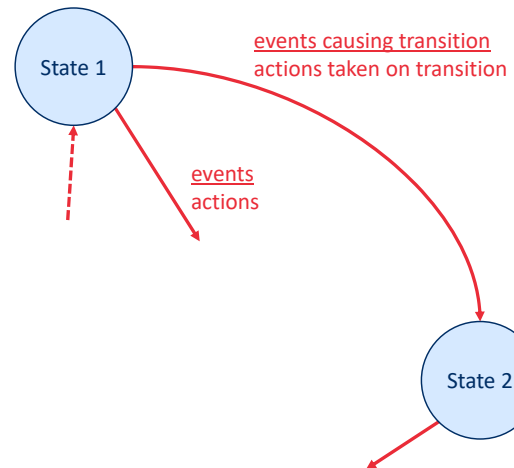
The service abstraction provided to the upper-layer entities is that of a reliable channel through which data can be transferred. With a reliable channel, no transferred data bits are corrupted (flipped from 0 to 1, or vice versa) or lost, and all are delivered in the order in which they were sent. This is precisely the service model offered by TCP to the Internet applications that invoke it.

It is the responsibility of a **reliable data transfer protocol** to implement this service abstraction. This task is made difficult by the fact that the layer *below* the reliable data transfer protocol may be unreliable. For example, TCP is a reliable data transfer protocol that is implemented on top of an unreliable (IP) end-to-end network layer. More generally, the layer beneath the two reliably communicating end points might consist of a single physical link (as in the case of a link-level data transfer protocol) or a global internetwork (as in the case of a transport-level protocol). For our purposes, however, we can view this lower layer simply as an unreliable point-to-point channel.

In the implementation of the reliable data service, the sending side of the data transfer protocol will be invoked from above by a call to `rdt_send()`. It will pass the data to be delivered to the upper layer at the receiving side. Here `rdt` stands for *reliable data transfer* protocol and `_send` indicates that the sending side of `rdt` is being called. On the receiving side, `rdt_rcv()` will be called when a packet arrives from the receiving side of the channel. When the `rdt` protocol wants to deliver data to the upper layer, it will do so by calling `deliver_data()`.

Finite state machines (FSM)

- We will represent protocols using finite state machines
 - Sender and receiver each have their own FSM
 - Start in an “initial state” marked by dashed arrow
 - Events cause a transition from one state to another
- Transition specifies action taken
 - Specified as events/actions
 - Empty event or action is marked by $\wedge$
 - For example: “receive packet/send acknowledgment”

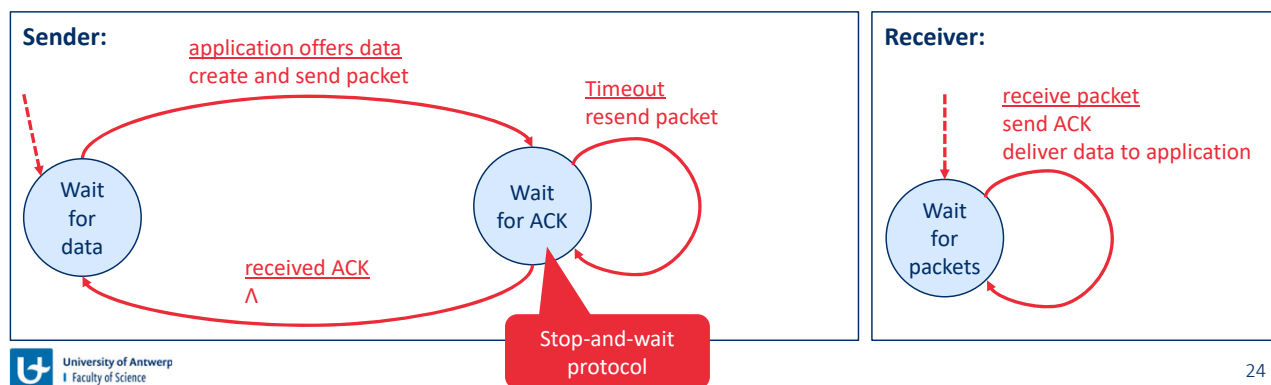


The finite-state machine (FSM) defines the operation of the sender or the receiver. It is important to note that there are *separate* FSMs for the sender and for the receiver. The arrows in the FSM description indicate the transition of the protocol from one state to another. The event causing the transition is shown above the horizontal line labelling the transition, and the actions taken when the event occurs are shown below the horizontal line. When no action is taken on an event, or no event occurs and an action is taken, we'll use the symbol $\wedge$ below or above the horizontal, respectively, to explicitly denote the lack of an action or event. The initial state of the FSM is indicated by the dashed arrow.

Automatic Repeat reQuest (ARQ) protocols

ARQ is based on 3 fundamental capabilities

- Error detection
- Receiver feedback (e.g., a positive ACK)
- Retransmission by sender (if the received packet did not arrive or had bit errors)



24

Consider how you yourself might dictate a long message over the phone. In a typical scenario, the message taker might say “OK” after each sentence has been heard, understood, and recorded. If the message taker hears a garbled sentence, you’re asked to repeat the garbled sentence. This message-dictation protocol uses both **positive acknowledgments** (“OK”) and **negative acknowledgments** (“Please repeat that.”). These control messages allow the receiver to let the sender know what has been received correctly, and what has been received in error and thus requires repeating. In a computer network setting, reliable data transfer protocols based on such retransmission are known as **ARQ (Automatic Repeat reQuest) protocols**. Fundamentally, three additional protocol capabilities are required in ARQ protocols to handle the presence of bit errors:

- **Error detection.** First, a mechanism is needed to allow the receiver to detect when bit errors have occurred. Recall from the previous section that UDP uses the Internet checksum field for exactly this purpose.
- **Receiver feedback.** Since the sender and receiver are typically executing on different end systems, possibly separated by thousands of miles, the only way for the sender to learn of the receiver’s view of the world (in this case, whether or not a packet was received correctly) is for the receiver to provide explicit feedback to the sender. The positive (ACK) and negative (NAK) acknowledgment replies in the message-dictation scenario are examples of such feedback. In principle, these packets need only be one bit long; for example, a 0 value could indicate a NAK and a value of 1 could indicate an ACK.
- **Retransmission.** A packet that is received in error at the receiver will be retransmitted by the sender.

The send side has two states. In the leftmost state, the send-side protocol is waiting for data

to be passed down from the upper layer. When this event occurs, the sender will create a packet containing the data to be sent, along with a packet checksum (for example, as discussed for the case of a UDP segment), and then send the packet via the network layer. In the rightmost state, the sender protocol is waiting for an ACK or a NAK packet from the receiver. If an ACK packet is received, the sender knows that the most recently transmitted packet has been received correctly and thus the protocol returns to the state of waiting for data from the upper layer. If a NAK is received or nothing is received within a timeout period, the protocol retransmits the last packet and waits for an ACK or NAK to be returned by the receiver in response to the retransmitted data packet.

It is important to note that when the sender is in the wait-for-ACK-or-NAK state, it *cannot* get more data from the upper layer; that will happen only after the sender receives an ACK and leaves this state. Thus, the sender will not send a new piece of data until it is sure that the receiver has correctly received the current packet. Because of this behaviour, such protocols are known as **stop-and-wait** protocols.

The proposed stop-and-wait protocol has a fundamental flaw

- What happens if an ACK packet gets corrupted or is lost?
- If an ACK is corrupt, the packet can be **resent**
 - This introduces **duplicate packets**
 - The receiver thus needs a way to identify retransmissions
 - This can be done by adding sequence numbers (stop-and-wait only requires a 1-bit sequence number!)
- The sender can detect a lost packet or ACK by using a **timeout**
 - If no response is received after the timeout, either the packet or ACK was lost in transmission
 - Timeout should be at least the duration of a worst-case round-trip-time (RTT)
 - Worst-case RTT is very hard to estimate in most real networks



Homework: Develop the FSM for a stop-and-wait sender and receiver with a 1-bit sequence number and timeouts to handle packet loss and corruption of packets and ACKs.

The simple stop-and-wait protocol may look as if it works but, unfortunately, it has a fatal flaw. In particular, we haven't accounted for the possibility that the ACK or NAK packet could be corrupted! Unfortunately, our slight oversight is not as innocuous as it may seem. Minimally, we will need to add checksum bits to ACK/NAK packets in order to detect such errors. The more difficult question is how the protocol should recover from errors in ACK or NAK packets. The difficulty here is that if an ACK or NAK is corrupted, the sender has no way of knowing whether or not the receiver has correctly received the last piece of transmitted data.

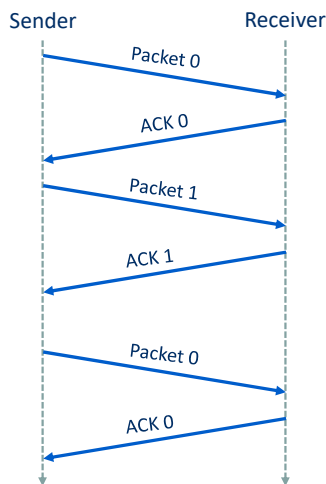
The sender can resend the current data packet when it receives a garbled ACK or NAK packet. This approach, however, introduces **duplicate packets** into the sender-to-receiver channel. The fundamental difficulty with duplicate packets is that the receiver doesn't know whether the ACK or NAK *it* last sent was received correctly at the sender. Thus, it cannot know *a priori* whether an arriving packet contains new data or is a retransmission!

A simple solution to this new problem (and one adopted in almost all existing data transfer protocols, including TCP) is to add a new field to the data packet and have the sender number its data packets by putting a **sequence number** into this field. The receiver then need only check this sequence number to determine whether or not the received packet is a retransmission. For this simple case of a stop-and-wait protocol, a 1-bit sequence number will suffice, since it will allow the receiver to know whether the sender is resending the previously transmitted packet (the sequence number of the received packet has the same sequence number as the most recently received packet) or a new packet (the sequence number changes, moving "forward" in modulo-2 arithmetic).

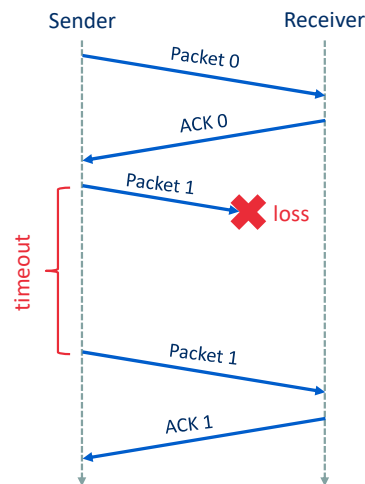
There are many possible approaches toward dealing with packet loss. Here, we'll put the burden of detecting and recovering from lost packets on the sender. Suppose that the sender transmits a data packet and either that packet, or the receiver's ACK of that packet, gets lost. In either case, no reply is forthcoming at the sender from the receiver. If the sender is willing to wait long enough so that it is *certain* that a packet has been lost, it can simply retransmit the data packet. You should convince yourself that this protocol does indeed work.

Stop-and-wait alternating-bit protocol

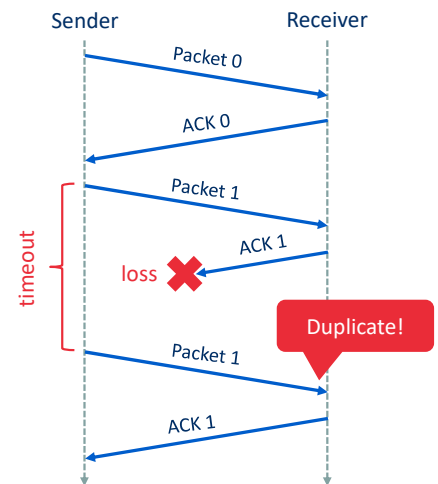
No packet loss:



Packet loss:

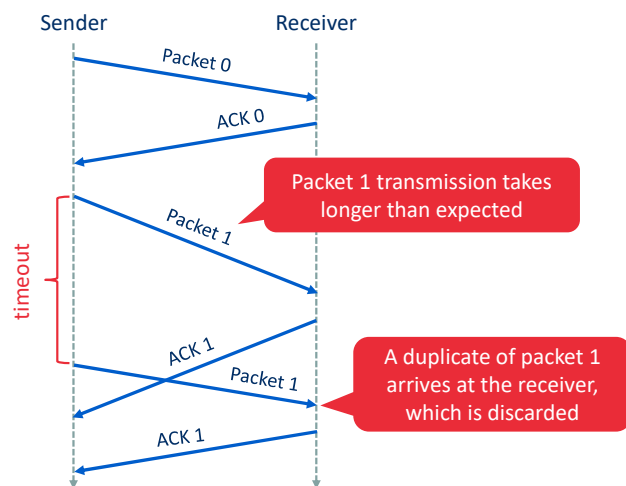


ACK loss:



The left figure shows how the protocol operates with no lost or delayed packets and how it handles lost data packets. Time moves forward from the top of the diagram toward the bottom of the diagram; note that a receive time for a packet is necessarily later than the send time for a packet as a result of transmission and propagation delays. The middle and right figure show what happens if a packet or ACK is lost. The timeout will expire, and the sender will retransmit the same packet (here shown as the packet with sequence number 1) again. Because packet sequence numbers alternate between 0 and 1, this protocol is sometimes known as the **alternating-bit protocol**.

What happens if RTT is higher than timeout?

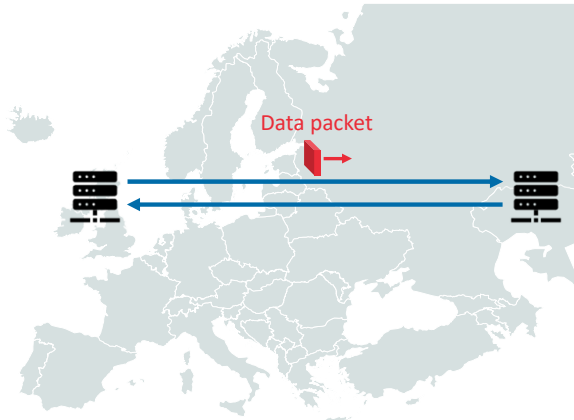


But how long must the sender wait to be certain that something has been lost? The sender must clearly wait at least as long as a round-trip delay between the sender and receiver (which may include buffering at intermediate routers) plus whatever amount of time is needed to process a packet at the receiver. In many networks, this worst-case maximum delay is very difficult even to estimate, much less know with certainty. Moreover, the protocol should ideally recover from packet loss as soon as possible; waiting for a worst-case delay could mean a long wait until error recovery is initiated. The approach thus adopted in practice is for the sender to judiciously choose a time value such that packet loss is likely, although not guaranteed, to have happened. If an ACK is not received within this time, the packet is retransmitted.

Stop-and-wait does not offer good performance

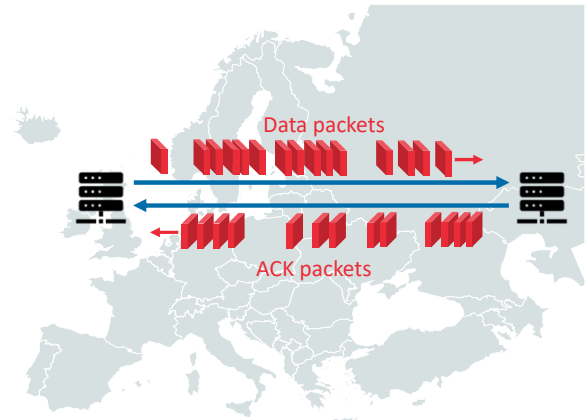
Stop-and-wait

Application layer throughput (goodput) depends solely on the RTT and the packet size



Pipelining

Needed for application goodput to approach link throughput



Exercise: Calculate sender utilization for stop-and-wait protocol



What is the sender utilization (fraction of time the sender is actively transmitting), assuming a 30msec RTT, a 1Gbps link throughput, a packet size (headers + data) of 1000 bytes, and an ACK size of 24 bytes?

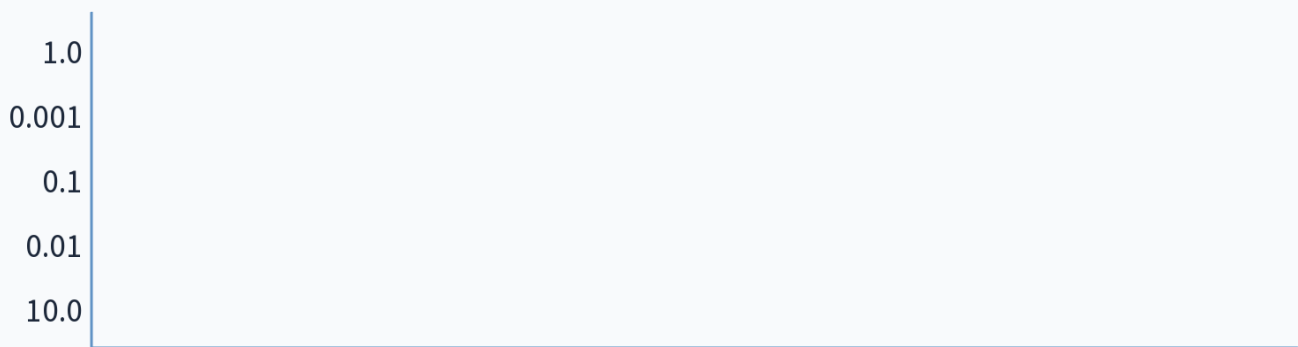
To appreciate the performance impact of this stop-and-wait behaviour, consider an idealized case of two hosts, one located in the UK and the other located on in Kazakhstan (distance about 4000 km). The speed-of-light round-trip propagation delay between these two end systems, RTT, is approximately 30 milliseconds. Suppose that they are connected by a channel with a transmission rate, R , of 1 Gbps (10^9 bits per second). With a packet size, L , of 1,000 bytes (8,000 bits) per packet, and an ACK size of 24 bytes, including both header fields and data.

Homework: Calculate sender utilization with pipelining



Given the same scenario as in the previous exercise. What would be the sender utilization in case 3-packet pipelining can be used (i.e., 3 unacknowledged packets can be transmitted)?

Suppose a packet is 10000 bits long, the channel transmission rate is 10Mbps, and the RTT is 9ms. What is the maximum channel utilization of a stop-and-wait protocol?



32

Start the presentation to see live content. For screen share software, share the entire screen. Get help at pollev.com/app

Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

Suppose a packet is 10000 bits long, the channel transmission rate is 10Mbps, and the RTT is 9ms. What is the maximum channel utilization of a stop-and-wait protocol?
https://www.polleverywhere.com/multiple_choice_polls/Dh6JnSegQM5hIzuRd1Ys6

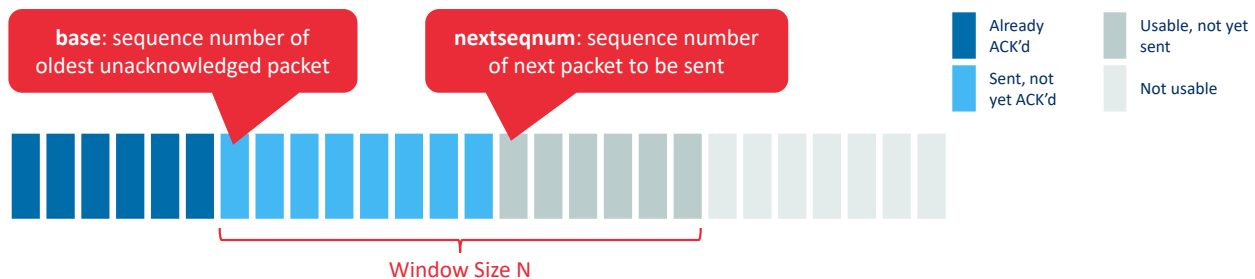
Suppose a 10000 bit packet , 10Mbps transmission rate , and 9ms RTT. What is the maximum channel utilization of a pipelined protocol?

- 1.0 **A**
- 0.001 **B**
- 0.1 **C**
- 0.01 **D**
- 10.0 **E**

Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

Suppose a 10000 bit packet , 10Mbps transmission rate , and 9ms RTT. What is the maximum channel utilization of a pipelined protocol?
https://www.polleverywhere.com/multiple_choice_polls/491rMWWDVny2IBVzE7Ifb?state=opened&flow=Default&onscreen=persist

Go-Back-N (GBN) sender



- **Cumulative ACKs:** $\text{ACK}(n)$ means all packets with sequence number $\leq n$ were successfully received
 - On receiving $\text{ACK}(n)$ move window forward and set $\text{base} = n + 1$
- A **timer** is set for oldest unacknowledged packet
 - On timeout, retransmit all unacknowledged packets
- At most N unacknowledged packets can be transmitted at any time

In a **Go-Back-N (GBN) protocol**, the sender is allowed to transmit multiple packets (when available) without waiting for an acknowledgment, but is constrained to have no more than some maximum allowable number, N , of unacknowledged packets in the pipeline. If we define *base* to be the sequence number of the oldest unacknowledged packet and *nextseqnum* to be the smallest unused sequence number (that is, the sequence number of the next packet to be sent), then four intervals in the range of sequence numbers can be identified. Sequence numbers in the interval $[0, \text{base}-1]$ correspond to packets that have already been transmitted and acknowledged. The interval $[\text{base}, \text{nextseqnum}-1]$ corresponds to packets that have been sent but not yet acknowledged. Sequence numbers in the interval $[\text{nextseqnum}, \text{base}+N-1]$ can be used for packets that can be sent immediately, should data arrive from the upper layer. Finally, sequence numbers greater than or equal to $\text{base}+N$ cannot be used until an unacknowledged packet currently in the pipeline (specifically, the packet with sequence number *base*) has been acknowledged.

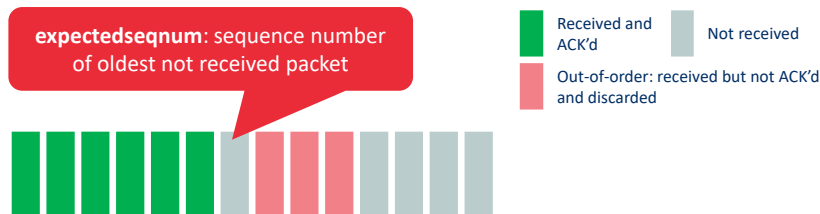
The range of permissible sequence numbers for transmitted but not yet acknowledged packets can be viewed as a window of size N over the range of sequence numbers. As the protocol operates, this window slides forward over the sequence number space. For this reason, N is often referred to as the **window size** and the GBN protocol itself as a **sliding-window protocol**. You might be wondering why we would even limit the number of outstanding, unacknowledged packets to a value of N in the first place. Why not allow an unlimited number of such packets? This is related to the flow and congestion control functionalities, which we will discuss later in the context of TCP.

In practice, a packet's sequence number is carried in a fixed-length field in the packet header.

If k is the number of bits in the packet sequence number field, the range of sequence numbers is thus $[0, 2^k - 1]$. With a finite range of sequence numbers, all arithmetic involving sequence numbers must then be done using modulo 2^k arithmetic.

In our GBN protocol, an acknowledgment for a packet with sequence number n will be taken to be a **cumulative acknowledgment**, indicating that all packets with a sequence number up to and including n have been correctly received at the receiver. The protocol's name, "Go-Back-N," is derived from the sender's behaviour in the presence of lost or overly delayed packets. As in the stop-and-wait protocol, a timer will again be used to recover from lost data or acknowledgment packets. If a timeout occurs, the sender resends *all* packets that have been previously sent but that have not yet been acknowledged. If an ACK is received but there are still additional transmitted but not yet acknowledged packets, the timer is restarted. If there are no outstanding, unacknowledged packets, the timer is stopped.

Go-Back-N (GBN) receiver

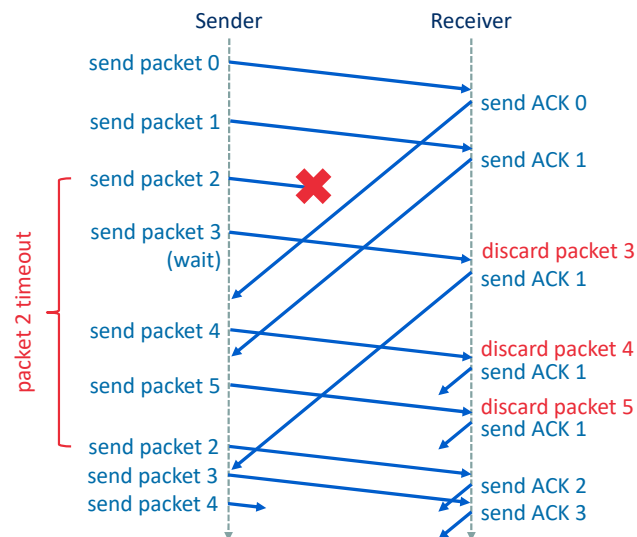


- When the packet with sequence number *expectedseqnum* is received
 - An ACK for that sequence number is sent and *expectedseqnum* is increased by 1
- When an out-of-order packet is received
 - The packet is discarded
 - Another ACK is sent with the highest in-order sequence number

The receiver's actions in GBN are also simple. If a packet with sequence number n is received correctly and is in order (that is, the data last delivered to the upper layer came from a packet with sequence number $n - 1$), the receiver sends an ACK for packet n and delivers the data portion of the packet to the upper layer. In all other cases, the receiver discards the packet and resends an ACK for the most recently received in-order packet. Note that since packets are delivered one at a time to the upper layer, if packet k has been received and delivered, then all packets with a sequence number lower than k have also been delivered. Thus, the use of cumulative acknowledgments is a natural choice for GBN.

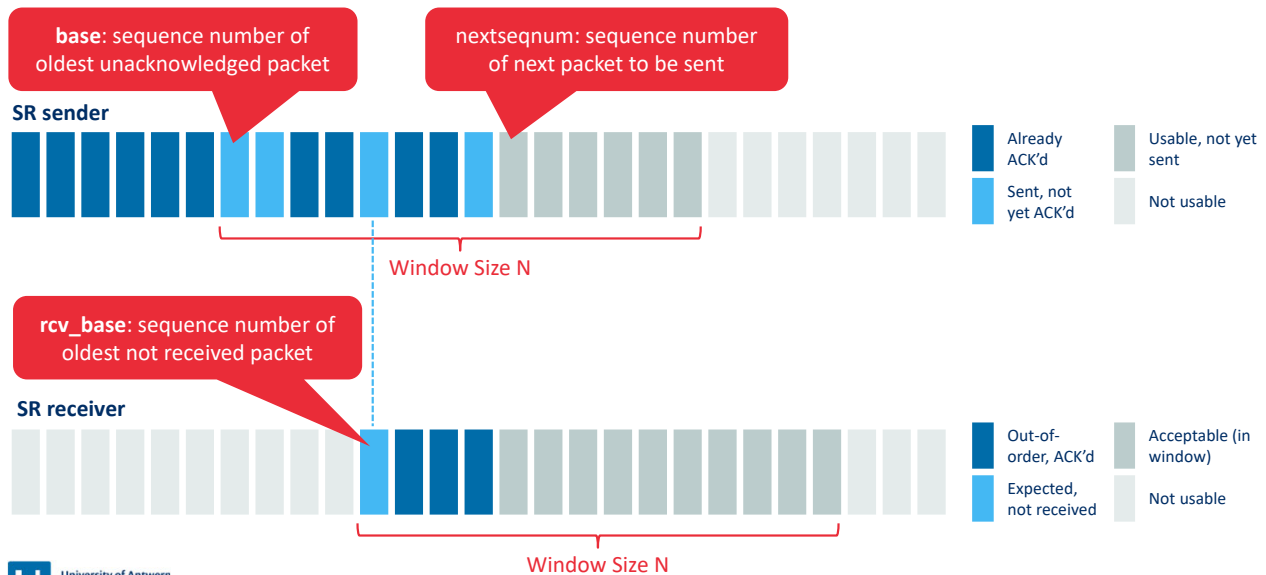
In our GBN protocol, the receiver discards out-of-order packets. Although it may seem silly and wasteful to discard a correctly received (but out-of-order) packet, there is some justification for doing so. Recall that the receiver must deliver data in order to the upper layer. Suppose now that packet n is expected, but packet $n + 1$ arrives. Because data must be delivered in order, the receiver *could* buffer (save) packet $n + 1$ and then deliver this packet to the upper layer after it had later received and delivered packet n . However, if packet n is lost, both it and packet $n + 1$ will eventually be retransmitted as a result of the GBN retransmission rule at the sender. Thus, the receiver can simply discard packet $n + 1$. The advantage of this approach is the simplicity of receiver buffering—the receiver need not buffer *any* out-of-order packets. Thus, while the sender must maintain the upper and lower bounds of its window and the position of *nextseqnum* within this window, the only piece of information the receiver need maintain is the sequence number of the next in-order packet. This value is held in the variable *expectedseqnum*. Of course, the disadvantage of throwing away a correctly received packet is that the subsequent retransmission of that packet might be lost or garbled and thus even more retransmissions would be required.

Example: Go-Back-4



The figure shows the operation of the GBN protocol for the case of a window size of four packets. Because of this window size limitation, the sender sends packets 0 through 3 but then must wait for one or more of these packets to be acknowledged before proceeding. As each successive ACK (for example, ACK0 and ACK1) is received, the window slides forward and the sender can transmit one new packet (pkt4 and pkt5, respectively). On the receiver side, packet 2 is lost and thus packets 3, 4, and 5 are found to be out of order and are discarded.

Selective Repeat (SR)

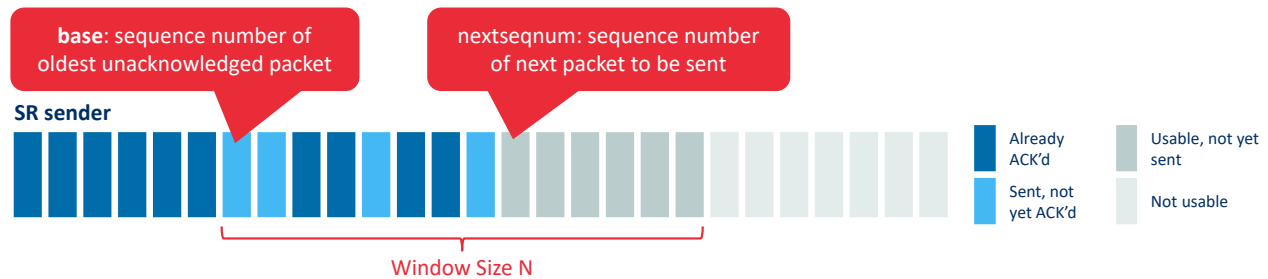


The GBN protocol allows the sender to potentially “fill the pipeline” with packets, thus avoiding the channel utilization problems we noted with stop-and-wait protocols. There are, however, scenarios in which GBN itself suffers from performance problems. In particular, when the window size and bandwidth-delay product are both large, many packets can be in the pipeline. A single packet error can thus cause GBN to retransmit a large number of packets, many unnecessarily. As the probability of channel errors increases, the pipeline can become filled with these unnecessary retransmissions.

As the name suggests, selective-repeat protocols avoid unnecessary retransmissions by having the sender retransmit only those packets that it suspects were received in error (that is, were lost or corrupted) at the receiver. This individual, as-needed, retransmission will require that the receiver *individually* acknowledge correctly received packets. A window size of N will again be used to limit the number of outstanding, unacknowledged packets in the pipeline. However, unlike GBN, the sender will have already received ACKs for some of the packets in the window.

The SR receiver will acknowledge a correctly received packet whether or not it is in order. Out-of-order packets are buffered until any missing packets (that is, packets with lower sequence numbers) are received, at which point a batch of packets can be delivered in order to the upper layer.

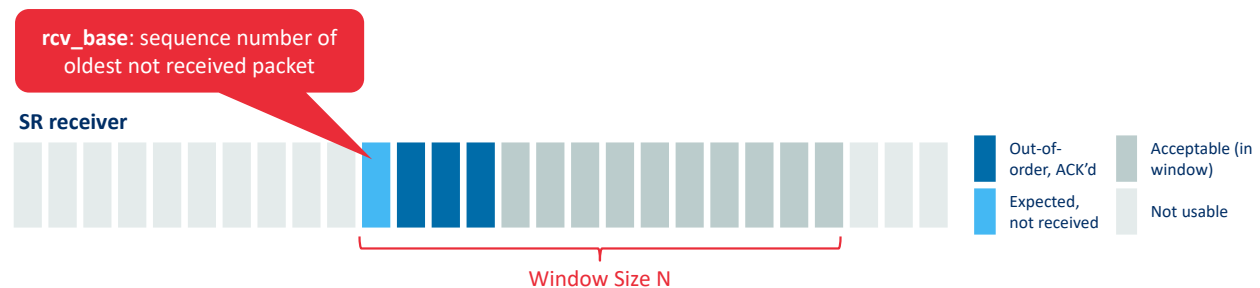
SR sender



- Data received from the application layer is only transmitted if the next available sequence number falls within the sender's window
- A logical timer is maintained for each packet, since only a single packet is retransmitted on a timeout
- If an ACK is received, the sender marks only that packet as received
 - If the sequence number equals send_base, the window moves to the next unacknowledged packet
 - Untransmitted packets that fall within the new window will then be transmitted

- **Data received from above.** When data is received from above, the SR sender checks the next available sequence number for the packet. If the sequence number is within the sender's window, the data is packetized and sent; otherwise it is either buffered or returned to the upper layer for later transmission, as in GBN.
- **Timeout.** Timers are again used to protect against lost packets. However, each packet must now have its own logical timer, since only a single packet will be transmitted on timeout. A single hardware timer can be used to mimic the operation of multiple logical timers.
- **ACK received.** If an ACK is received, the SR sender marks that packet as having been received, provided it is in the window. If the packet's sequence number is equal to send_base, the window base is moved forward to the unacknowledged packet with the smallest sequence number. If the window moves and there are untransmitted packets with sequence numbers that now fall within the window, these packets are transmitted.

SR receiver

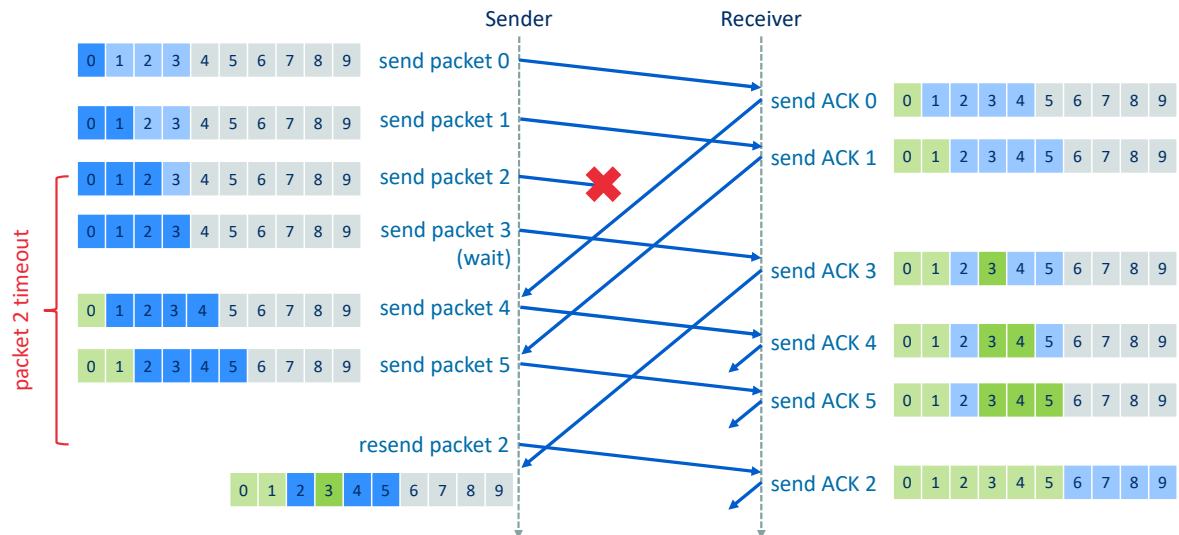


- If a packet arrives within the receive window, a selective ACK is returned and it is buffered
 - If the sequence number equals rcv_base, then it and all consecutively numbered buffered packets are delivered to the upper layer, and the receive window is moved forward by the number of delivered packets
- If a packet with sequence number [rcv_base - N, rcv_base - 1] is received, an ACK is generated
- Otherwise: Discard the packet

- **Packet with sequence number in [rcv_base, rcv_base + N - 1] is correctly received.** In this case, the received packet falls within the receiver's window and a selective ACK packet is returned to the sender. If the packet was not previously received, it is buffered. If this packet has a sequence number equal to the base of the receive window (rcv_base), then this packet, and any previously buffered and consecutively numbered (beginning with rcv_base) packets are delivered to the upper layer. The receive window is then moved forward by the number of packets delivered to the upper layer.
- **Packet with sequence number in [rcv_base - N, rcv_base - 1] is correctly received.** In this case, an ACK must be generated, even though this is a packet that the receiver has previously acknowledged.
- **Otherwise.** Ignore the packet.

It is important to note that in Step 2, the receiver reacknowledges (rather than ignores) already received packets with certain sequence numbers *below* the current window base. You should convince yourself that this reacknowledgment is indeed needed.

Example: SR with N = 4



How many distinct sequence numbers are needed?

- The window size N must be less than or equal to half the sequence number space
 - This is required in the corner case where all N ACKs of sender window packets are lost
 - Otherwise it might not be possible to differentiate between a new or retransmitted packet

Example

- Window size = 3, and sequence number space = 4
- Scenario 1
 - Packets 0, 1, and 2 are transmitted and received, but all 3 ACKs are lost
 - Senders will retransmit Packet 0 after timeout
- Scenario 2
 - Packets 0, 1, and 2 are transmitted, received, and ACK'd
 - Packet 3 is transmitted but lost after ACK 0
 - Packet 4 (with sequence number 0) is transmitted
- With only 4 sequence numbers, receiver cannot differentiate between Packet 0 retransmission in scenario 1 and Packet 4 transmission in scenario 2 (as both have sequence number 0)

The lack of synchronization between sender and receiver windows has important consequences when we are faced with the reality of a finite range of sequence numbers. Consider what could happen, for example, with a finite range of four packet sequence numbers, 0, 1, 2, 3, and a window size of three. Suppose packets 0 through 2 are transmitted and correctly received and acknowledged at the receiver. At this point, the receiver's window is over the fourth, fifth, and sixth packets, which have sequence numbers 3, 0, and 1, respectively. Now consider two scenarios. In the first scenario, the ACKs for the first three packets are lost and the sender retransmits these packets. The receiver thus next receives a packet with sequence number 0—a copy of the first packet sent. In the second scenario, the ACKs for the first three packets are all delivered correctly. The sender thus moves its window forward and sends the fourth, fifth, and sixth packets, with sequence numbers 3, 0, and 1, respectively. The packet with sequence number 3 is lost, but the packet with sequence number 0 arrives—a packet containing *new* data.

Now consider the receiver's viewpoint, which has a figurative curtain between the sender and the receiver, since the receiver cannot "see" the actions taken by the sender. All the receiver observes is the sequence of messages it receives from the channel and sends into the channel. As far as it is concerned, the two scenarios are *identical*. There is no way of distinguishing the retransmission of the first packet from an original transmission of the fifth packet. Clearly, a window size that is 1 less than the size of the sequence number space won't work. But how small must the window size be? The window size must be less than or equal to half the size of the sequence number space for SR protocols.

Given a window size $N = 5$ and a selective repeat transmitter and receiver. How many bits should at least be used to store the sequence number?

- 3 **A**
- 4 **B**
- 5 **C**
- 10 **D**
- 11 **E**

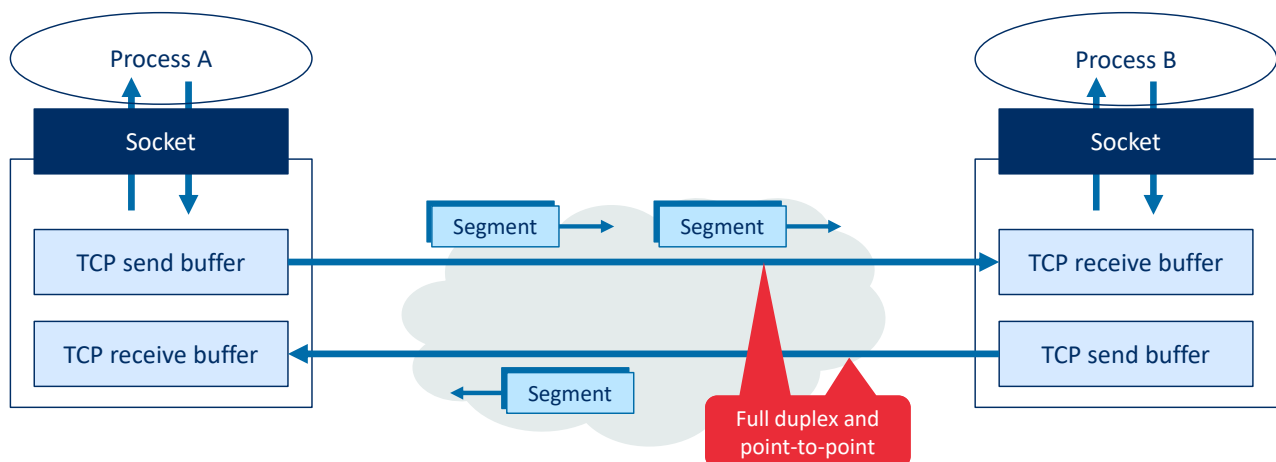
Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

Given a window size $N = 5$ and a selective repeat transmitter and receiver. How many bits should at least be used to store the sequence number?
https://www.polleverywhere.com/multiple_choice_polls/nsW3YAOaQW7S7pAH2AmDD?state=opened&flow=Default&onscreen=persist

Transport Control Protocol (TCP)

Let's turn to TCP—the Internet's transport-layer, connection-oriented, reliable transport protocol. We'll see that in order to provide reliable data transfer, TCP relies on many of the underlying principles discussed in the previous section, including error detection, retransmissions, cumulative acknowledgments, timers, and header fields for sequence and acknowledgment numbers. TCP is defined in RFC 9293 (newer version of RFC 793), RFC 1122. Principles of congestion control are described in RFC 5681. TCP extensions, such as timestamps and selective acknowledgement, are described in RFC 7323 and RFC 2018. TCP NewReno is described in RFC 6582. The most popular TCP congestion avoidance algorithm (TCP Cubic), which is default in many operating systems, is defined in RFC 9438.

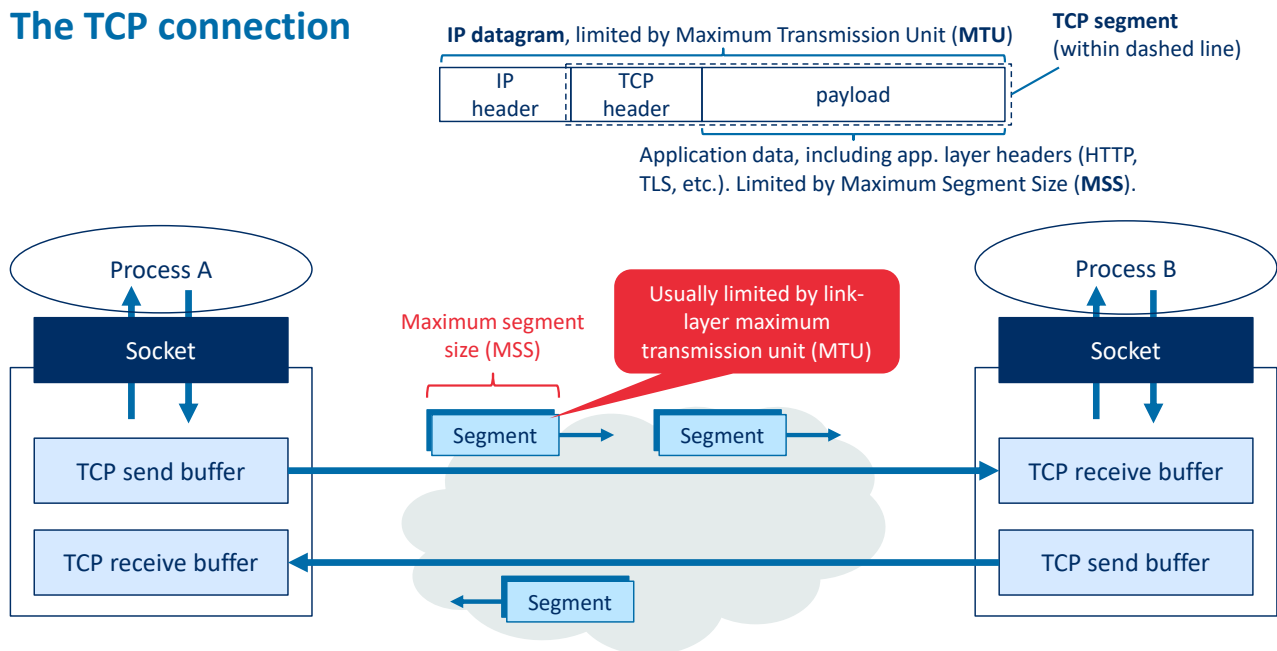
The TCP connection



A TCP connection provides a **full-duplex service**: If there is a TCP connection between Process A on one host and Process B on another host, then application-layer data can flow from Process A to Process B using the same TCP connection as application-layer data from Process B to Process A. A TCP connection is also always **point-to-point**, that is, between a single sender and a single receiver. So-called “multicasting”—the transfer of data from one sender to many receivers in a single send operation—is not possible with TCP. With TCP, two hosts are company and three are a crowd!

Once a TCP connection is established, the two application processes can send data to each other. Let’s consider the sending of data from Process A to Process B. Process A passes a stream of data through the socket (the “door” of the process). Once the data passes through the socket, the data is in the hands of TCP running in the client’s operating system. TCP directs this data to the connection’s **send buffer**, which is one of the buffers that is set aside during the initial three-way handshake. From time to time, TCP will grab chunks of data from the send buffer and pass the data to the network layer. TCP specification [RFCs 9293, 793] gives some recommendations when TCP should send data based on the amount of buffered data, number of packets that can be pipelined (similar to Go-Back-N), and available **receive buffer** size on receiver’s side.

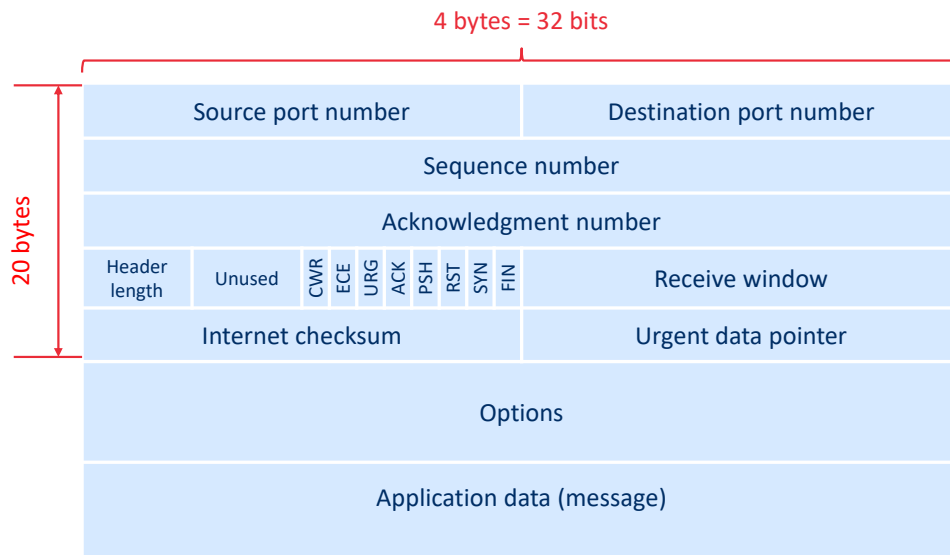
The TCP connection



The maximum amount of data that can be grabbed and placed in a segment is limited by the **maximum segment size (MSS)**. The MSS is typically set by first determining the length of the largest IP datagram that can be sent by the local sending host (the so-called **maximum transmission unit, MTU**) and then setting the MSS to ensure that payload plus TCP and IP headers length (minimum 20 bytes for TCP and 20 bytes for IPv4) will fit into MTU. The value of MTU varies depending on the technology used at link layer. For example, Ethernet link-layer protocol has an MTU of 1500 bytes. Thus, a maximum value of MSS in this case is 1460 bytes. TCP pairs each chunk of client data with a TCP header, thereby forming **TCP segments**. The segments are passed down to the network layer, where they are separately encapsulated within network-layer IP datagrams. The IP datagrams are then sent into the network.

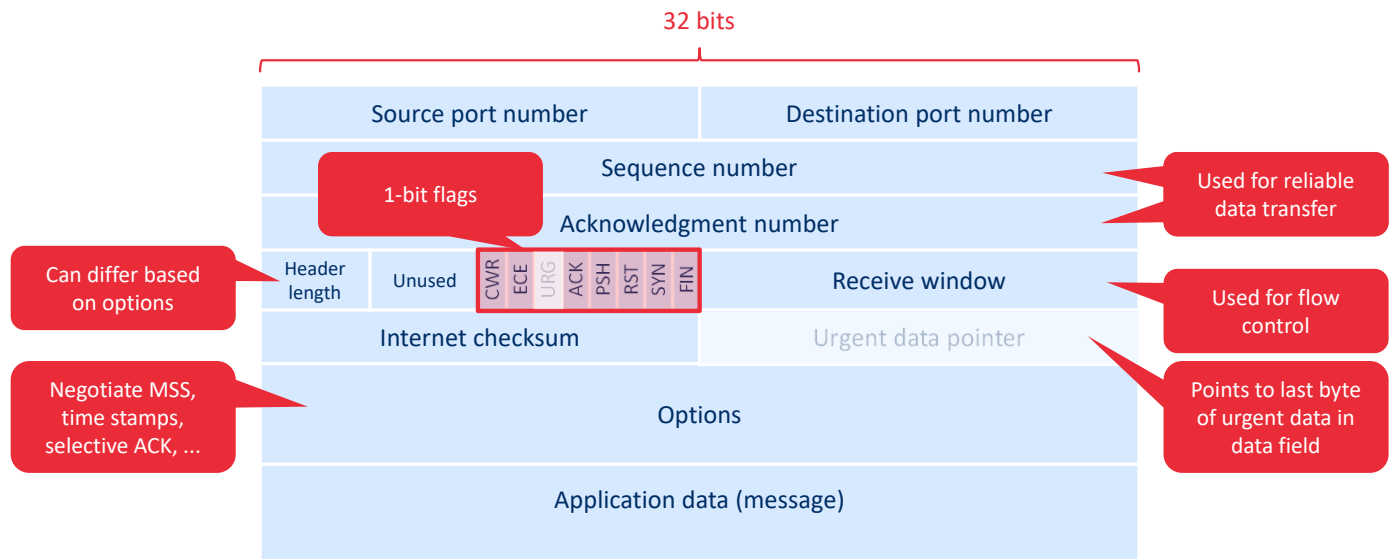
After decapsulation at the other end, TCP receives a segment and analyses if it is received correctly and in-order. If the order is correct, the data from this segment is placed into the **receive buffer**, from which the application can read the data. Otherwise, the segment is kept apart for further reassembling once the gap is filled. Each side of the connection has its own send buffer and its own receive buffer.

TCP segment structure



The TCP segment consists of TCP header and TCP payload containing a chunk of application data. As mentioned earlier, the MSS limits the maximum size of a segment's data field. When TCP sends a large file, such as an image on a Web page, it typically breaks the file into chunks of size MSS (except for the last chunk, which will often be less than the MSS). Interactive applications, however, often transmit data chunks that are smaller than the MSS; for example, with remote login applications such as Telnet and ssh, the data field in the TCP segment is often only one byte.

TCP segment structure



As with UDP, the header includes **source and destination port numbers**, which are used for multiplexing/ demultiplexing data from/to upper-layer applications. Also, as with UDP, TCP header includes a **checksum field**. A TCP segment header also contains the following fields:

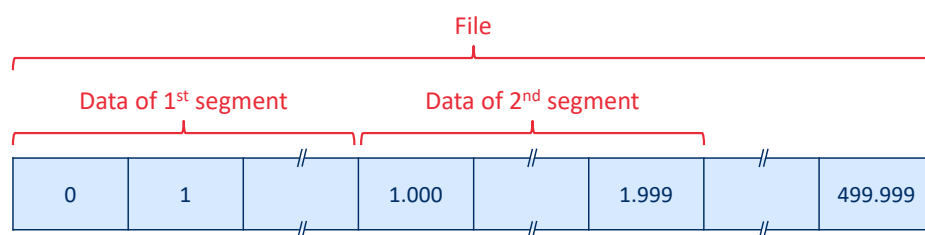
- The 32-bit **sequence number field** and the 32-bit **acknowledgment number field** are used by the TCP sender and receiver in implementing a reliable data transfer service. Since acknowledgement number is always present in the header, the same packet can simultaneously confirm the data reception and contain new data.
- The 16-bit **receive window field** is used for flow control. We will see that it is used to indicate the number of bytes that a receiver is willing to accept.
- The 4-bit **header length field** specifies the length of the TCP header in 32-bit words. The TCP header can be of variable length due to the TCP options field. The minimum size of a TCP header is 20 bytes. However, typically a few options are present (e.g., timestamps, selective acknowledgement).
- The optional and variable-length **options field** is used for timestamps (used for RTT estimation), selective acknowledgements, maximum segment size (MSS) negotiation, window scaling factor in high-speed networks, etc.
- The **flags field** contains 8 bits.
 - The **ACK bit** is used to indicate that the value carried in the acknowledgment field is valid; that is, the segment contains an acknowledgment for a segment that has been successfully received.
 - The **RST**, **SYN**, and **FIN** bits are used for connection setup and teardown.
 - The **CWR** and **ECE** bits are used in explicit congestion notification, which we will discuss later in this lecture.
 - Setting the **PSH** bit indicates that the receiver should pass the data to the

application, i.e., place in the socket buffer, immediately. In case PSH flag is not set, TCP receiver can keep the data (received correctly and in-order) in the receive buffer and wait for one or a few more segments in order to place a bigger chunk in the socket buffer at once. For example, when a long web page is transmitted in multiple TCP segments, PSH flag can be set in the last segment to push the full page to the application. Note that even with PSH flag the data cannot be pushed to application out of order. In practice, handling of PSH flag can vary for different implementations.

- Finally, the **URG** bit is used to indicate that there is data in this segment that the sending-side application has marked as “urgent.” The location of the last byte of this urgent data is indicated by the 16-bit **urgent data pointer field**. TCP must inform the receiving-side upper-layer entity when urgent data exists and pass it a pointer to the end of the urgent data. In practice, usage of URG flag is deprecated. RFC 9293 specifically states that “new applications **SHOULD NOT** set the URG flag due to implementation differences and middlebox issues”.

Sequence and acknowledgement numbers

- TCP views data as an unstructured, but ordered, stream of bytes
- **Sequence number** of a segment refers to the position of its first byte in the stream and **NOT** the number of the segment itself
- **Acknowledgement number** is the next byte that receiver expects from the sender
 - TCP thus uses **cumulative acknowledgments**
- Example (MSS=1000 bytes, File = 500.000 bytes):



TCP views data as an unstructured, but ordered, stream of bytes. TCP's use of sequence numbers reflects this view in that sequence numbers are over the stream of transmitted bytes and *not* over the series of transmitted segments. The **sequence number for a segment** is therefore the byte-stream number of the first byte in the segment. Let's look at an example. Suppose that a process in Host A wants to send a stream of data to a process in Host B over a TCP connection. The TCP in Host A will implicitly number each byte in the data stream. Suppose that the data stream consists of a file consisting of 500,000 bytes, that the MSS is 1,000 bytes, and that the first byte of the data stream is numbered 0. As shown in the Figure, TCP constructs 500 segments out of the data stream. The first segment gets assigned sequence number 0, the second segment gets assigned sequence number 1,000, the third segment gets assigned sequence number 2,000, and so on. Each sequence number is inserted in the sequence number field in the header of the appropriate TCP segment.

Now let's consider acknowledgment numbers. These are a little trickier than sequence numbers. Recall that TCP is full-duplex, so that Host A may be receiving data from Host B while it sends data to Host B (as part of the same TCP connection). Each of the segments that arrive from Host B has a sequence number for the data flowing from B to A. *The acknowledgment number that Host A puts in its segment is the sequence number of the next byte Host A is expecting from Host B.* It is good to look at a few examples to understand what is going on here. Suppose that Host A has received all bytes numbered 0 through 535 from B and suppose that it is about to send a segment to Host B. Host A is waiting for byte 536 and all the subsequent bytes in Host B's data stream. So, Host A puts 536 in the acknowledgment number field of the segment it sends to B. Because TCP only acknowledges bytes up to the first missing byte in the stream, TCP is said to provide **cumulative acknowledgments**.

This last example also brings up an important but subtle issue. Host A received the third segment (bytes 900 through 1,000) before receiving the second segment (bytes 536 through 899). Thus, the third segment arrived out of order. There are basically two choices here: either (1) the receiver immediately discards out-of-order segments (which, as we discussed earlier, can simplify receiver design), or (2) the receiver keeps the out-of-order bytes and waits for the missing bytes to fill in the gaps. TCP RFC 1122 states that “TCP SHOULD be capable of queueing out-of-order TCP segments”. Clearly, this choice is more efficient, and this is the approach taken in practice.

Host A has just successfully received a TCP segment with sequence number 12000. Assuming an MSS of 1000, what acknowledgement number does Host A send back?

(A) 12000 0

(B) 12001 0

(C) 12999 0

(D) 13000 0

Start the presentation to see live content. For screen share software, share the entire screen. Get help at pollev.com/app

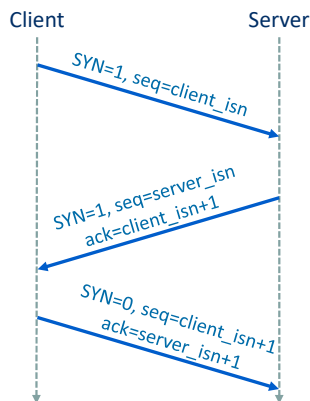
Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.

More info at polleverywhere.com/support

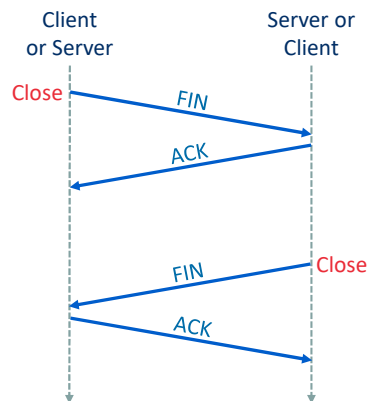
Host A has just successfully received a TCP segment with sequence number 12000. Assuming an MSS of 1000, what acknowledgement number does Host A send back?
https://www.polleverywhere.com/multiple_choice_polls/rjJnFWGh5YQtoxvDKrfVO?sate=opened&flow=Default&onscreen=persist

Connection management

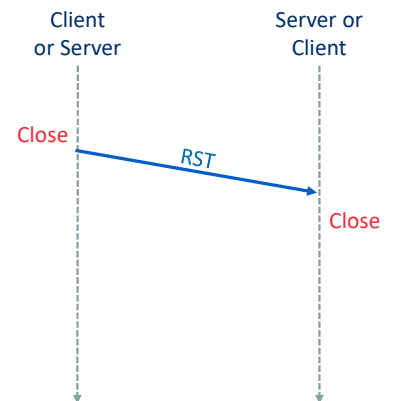
Connection establishment (three-way handshake)



Closing the TCP connection gracefully



Aborting the TCP connection



Let's first take a look at how a TCP connection is established. Suppose a process running in one host (client) wants to initiate a connection with another process in another host (server). The client application process first informs the client TCP that it wants to establish a connection to a process in the server. The TCP in the client then proceeds to establish a TCP connection with the TCP in the server in the following manner:

- **Step 1.** The client-side TCP first sends a special **SYN segment** to the server-side TCP. This special segment contains no application-layer data. But one of the flag bits in the segment's header, the SYN bit, is set to 1. For this reason, this special segment is referred to as a SYN segment. In addition, the client randomly chooses an initial sequence number (client_isn) and puts this number in the sequence number field of the initial TCP SYN segment. This segment is encapsulated within an IP datagram and sent to the server.
- **Step 2.** Once the IP datagram containing the TCP SYN segment arrives at the server host (assuming it does arrive!), the server extracts the TCP SYN segment from the datagram, and sends a connection-granted segment to the client TCP. This connection-granted segment also contains no application-layer data. However, it does contain three important pieces of information in the segment header. First, the SYN bit is set to 1. Second, the acknowledgment field of the TCP segment header is set to client_isn+1. Finally, the server chooses its own initial sequence number (server_isn) and puts this value in the sequence number field of the TCP segment header. This connection-granted segment is saying, in effect, "I received your SYN packet to start a connection with your initial sequence number, client_isn. I agree to establish this connection. My own initial sequence number is server_isn." The connection-granted segment is referred to as a **SYNACK segment**.

- **Step 3.** Upon receiving the SYNACK segment, the client also allocates buffers and variables to the connection. The client host then sends the server yet another segment, **ACK segment**; this last segment acknowledges the server's connection-granted segment (the client does so by putting the value `server_isn+1` in the acknowledgment field of the TCP segment header). The SYN bit is set to zero, since the connection is established. This third segment may carry client-to-server data in the segment payload.
- **Step 4.** Upon receiving the ACK segment from the client, the server allocates resources for buffers and variables reflecting the state of the connection. It could also be done at Step2, but then server becomes vulnerable to SYN flooding DDOS attacks. A technique called SYN cookie is used to verify this last ACK segment without need to store any information at Step 2 [RFC 4987].

Once these three steps have been completed, the client and server hosts can send segments containing data to each other. In each of these future segments, the SYN bit will be set to zero. Note that in order to establish the connection, three packets are sent between the two hosts. For this reason, this connection-establishment procedure is often referred to as a **three-way handshake**.

All good things must come to an end, and the same is true with a TCP connection. Either of the two processes participating in a TCP connection can end the connection. When a connection ends, the "resources" (that is, the buffers and variables) in the hosts are deallocated. As an example, suppose the client decides to close the connection. The client application process issues a close command. This causes the client TCP to send a special **FIN segment** to the server process. This special segment has a flag bit in the segment's header, the FIN bit, set to 1. When the server receives this segment, it sends the client an acknowledgment segment in return. Now the connection is half-closed, and the server can still send data to the client, and the client can acknowledge this data. When the server is ready, it sends its own shutdown segment, which has the FIN bit set to 1. Finally, the client acknowledges the server's shutdown segment. At this point, all the resources in the two hosts are deallocated.

At any point when the TCP end-point decides to abort the connection, e.g., due to very high packet loss or an unexpected packet, it can always send a TCP segment with **RST** bit set to 1. In this case, the initiator of RST segment will no longer send and receive data and will not wait for an acknowledgement. The receiving end-point will verify the RST segments using Sequence number and Acknowledgement number fields before reacting to it and aborting the connection.

Estimating the round-trip time (RTT)

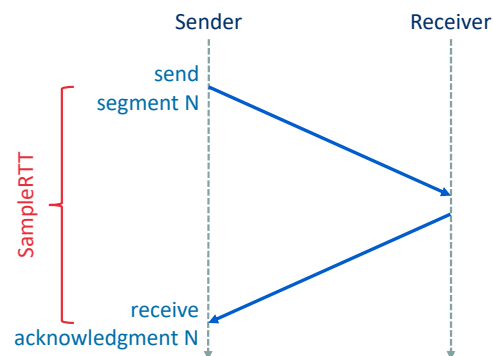
- The TCP retransmit timeout should be larger than the RTT
- How to estimate the RTT in a real network?
 - Measured as the time between sending a segment and receiving the corresponding acknowledgment (**SampleRTT**)
 - Never calculated for segments that are retransmitted
 - To reduce fluctuations, **EstimatedRTT** is an exponential weighted moving average (EWMA) of SampleRTT
 - The variability of the RTT is calculated as **DevRTT**

Recommendation:
 $\alpha = 0.125$

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

Recommendation:
 $\beta = 0.25$



TCP uses a timeout/retransmit mechanism to recover from lost segments. Although this is conceptually simple, many subtle issues arise when we implement a timeout/retransmit mechanism in an actual protocol such as TCP. Perhaps the most obvious question is the length of the timeout intervals. Clearly, the timeout should be larger than the connection's round-trip time (RTT), that is, the time from when a segment is sent until it is acknowledged. Otherwise, unnecessary retransmissions would be sent. But how much larger? How should the RTT be estimated in the first place?

Let's begin our study of TCP timer management by considering how TCP estimates the round-trip time between a sender and a receiver. The sample RTT, denoted **SampleRTT**, for a segment is the amount of time between when the segment is sent (that is, passed to IP) and when an acknowledgment for the segment is received. When an acknowledgement for a retransmitted segment is received, it is **not** used for RTT measurement, because it is not clear which packet triggered this acknowledgement, original one or the retransmission. The sender remembers the sent time of the oldest unacknowledged segment only. When the acknowledgement for this segment is received, SampleRTT is calculated, and the new sent time is remembered for new data. This way, a new value of SampleRTT is calculated once every RTT.

Obviously, the SampleRTT values will fluctuate from segment to segment, e.g., due to congestion in the routers, or varying load on the end systems. Because of this fluctuation, any given SampleRTT value may be atypical. In order to estimate a typical RTT, it is therefore natural to take some sort of average of the SampleRTT values. TCP maintains an average, called EstimatedRTT, of the SampleRTT values. Upon obtaining a new SampleRTT, TCP

updates **EstimatedRTT** according to the following formula:

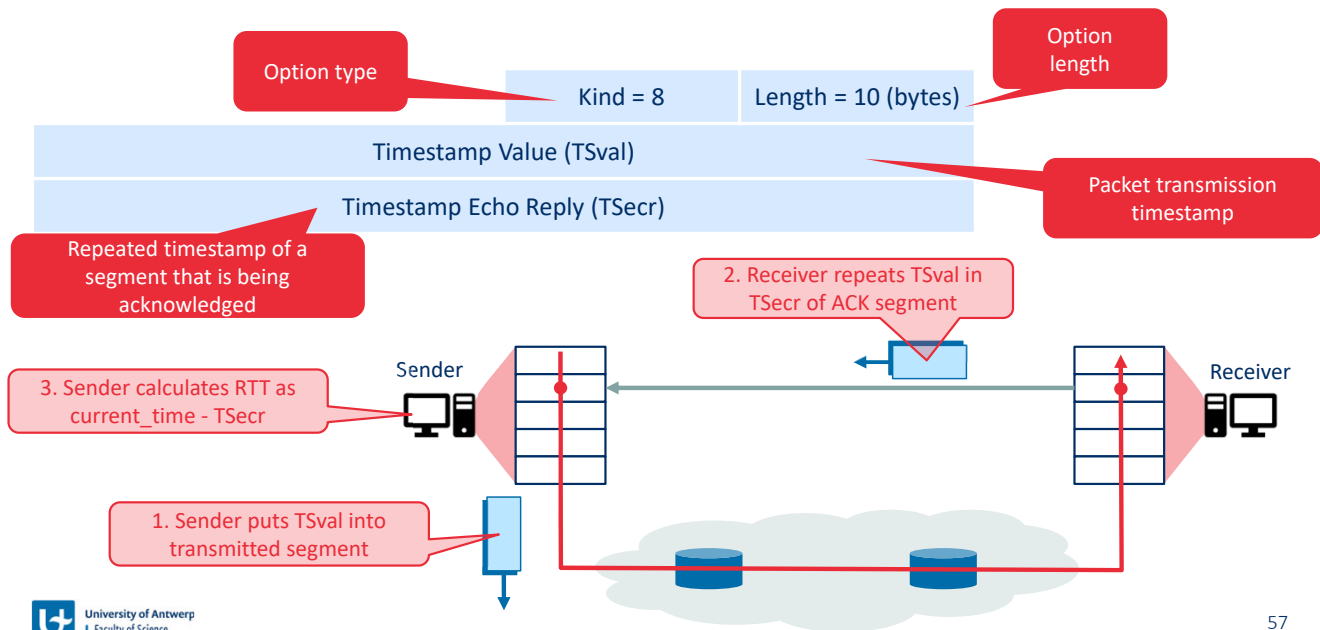
$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

Note that EstimatedRTT is a weighted average of the SampleRTT values. This weighted average puts more weight on recent samples than on old samples. This is natural, as the more recent samples better reflect the current congestion in the network. In statistics, such an average is called an **exponential weighted moving average (EWMA)**. In addition to having an estimate of the RTT, the variability of the RTT is measured. [RFC 6298] defines the RTT variation, **DevRTT**, as an estimate of how much SampleRTT typically deviates from EstimatedRTT:

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

The recommended value of α is 0.125 and of β is 0.25.

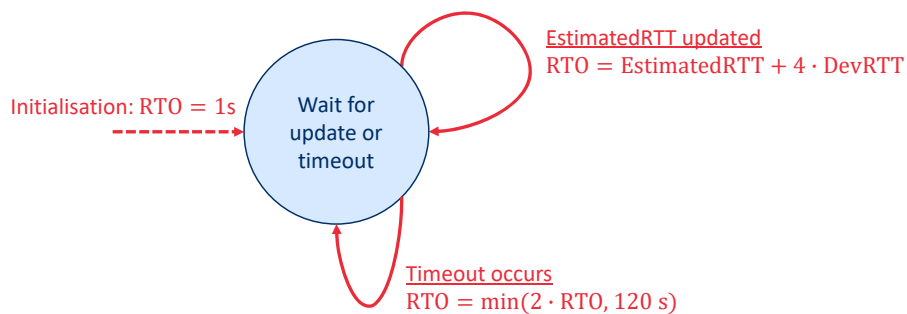
Estimating the round-trip time (RTT) using Timestamp Option



A common issue with RTT estimation is ignorance of retransmitted segments due to ambiguity: it is not clear whether the ACK was received in reply of the originally transmitted segment, or in reply to a retransmission. With TCP timestamp options, the receiver can indicate the transmission timestamp of the segment it replies to. It works as follows. When TCP sender transmits a segment, it indicates the timestamp in TSval field of the Timestamp Option. When TCP receiver replies with the ACK, it repeats this timestamp in TSecr field of Timestamp Option. Once TCP sender receives ACK, it reads the timestamp from TSecr, and calculates RTT as a difference between the reception time and the timestamp indicated in TSecr.

Managing the retransmission timeout (RTO)

- RTO should be close to but larger than EstimatedRTT
 - Margin should be larger if the variability (DevRTT) is large
 - An initial RTO = 1 second is recommended
 - When a timeout occurs, the RTO is immediately doubled, but recalculated when EstimatedRTT is next updated

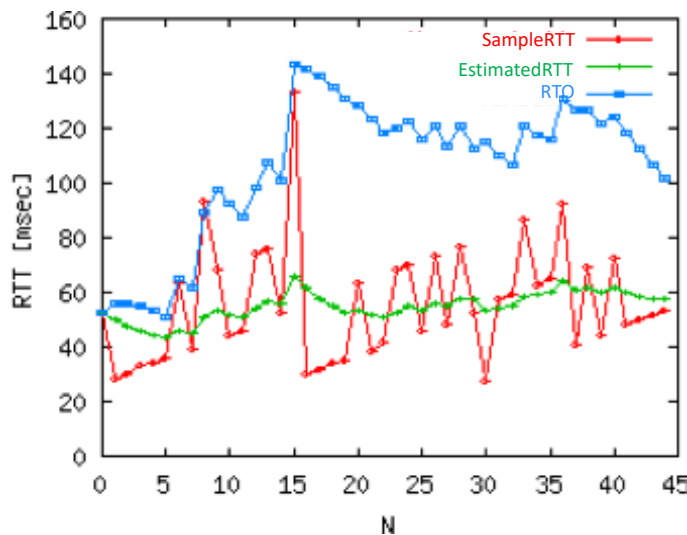


Given values of EstimatedRTT and DevRTT, what value should be used for TCP's timeout interval? Clearly, the interval should be greater than or equal to EstimatedRTT, or unnecessary retransmissions would be sent. But the timeout interval should not be too much larger than EstimatedRTT; otherwise, when a segment is lost, TCP would not quickly retransmit the segment, leading to large data transfer delays. It is therefore desirable to set the timeout equal to the EstimatedRTT plus some margin. The margin should be large when there is a lot of fluctuation in the SampleRTT values; it should be small when there is little fluctuation. The value of DevRTT should thus come into play here. All of these considerations are taken into account in TCP's method for determining the retransmission timeout (**RTO**) interval:

$$RTO = EstimatedRTT + 4 \cdot DevRTT$$

An initial RTO value of 1 second is recommended [RFC 6298]. Also, when a timeout occurs, the value of RTO is doubled to avoid a premature timeout occurring for subsequent segments. However, as soon as an acknowledgement for a non-retransmitted segment is received and EstimatedRTT is updated, the RTO value is again computed using the formula above.

Example of sampleRTT, estimated RTT, and RTO over time



$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

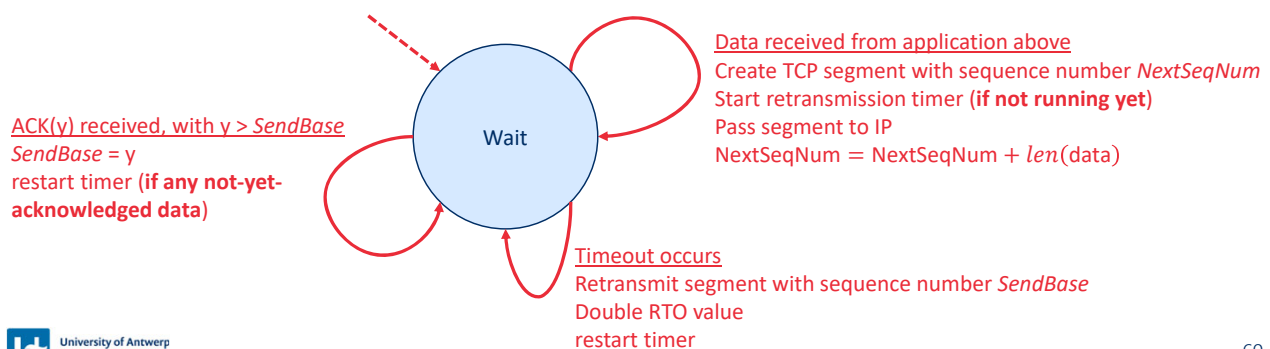
$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

$$\text{RTO} = \min(\text{EstimatedRTT} + 4 \cdot \text{DevRTT}, 120\text{s})$$

The graph shows the SampleRTT, EstimatedRTT and RTO values over time. Clearly, the variations in the SampleRTT are smoothed out in the computation of the EstimatedRTT.
Source: <https://beta.computer-networking.info/syllabus/default/protocols/tcp.html#tcp-s-retransmission-timeout>

Retransmission timer management

- RFC 6298 recommends the TCP sender keeps a **single** retransmission timer
 - Associated with the oldest unacknowledged segment
 - Restarts when an acknowledgment for this segment is received
 - When the timer expires sender starts retransmission of the oldest unacknowledged segment
- Simplified TCP sender:



In our earlier development of reliable data transfer techniques, it was conceptually easiest to assume that an individual timer is associated with each transmitted but not yet acknowledged segment. While this is great in theory, timer management can require considerable overhead. Thus, the recommended TCP timer management procedures [RFC 6298] use only a *single* retransmission timer, even if there are multiple transmitted but not yet acknowledged segments. The TCP protocol described here follows this single-timer recommendation.

There are three major events related to data transmission and retransmission in the TCP sender: data received from application above; timer timeout; and ACK receipt. Upon the occurrence of the first major event, TCP receives data from the application, encapsulates the data in a segment, and passes the segment to IP. Note that each segment includes a sequence number that is the byte-stream number of the first data byte in the segment. Also note that if the timer is already not running for some other segment, TCP starts the timer when the segment is passed to IP. (It is helpful to think of the timer as being associated with the oldest unacknowledged segment.) The expiration interval for this timer is the RTO, which is calculated from EstimatedRTT and DevRTT according to Slides 56-57.

The second major event is the timeout. TCP responds to the timeout event by retransmitting the segment that caused the timeout, and by doubling the value of RTO. TCP then restarts the timer.

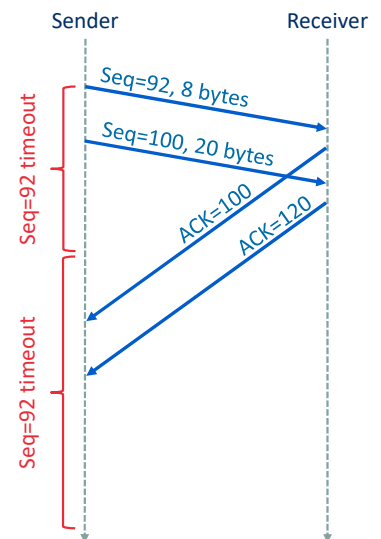
The third major event that must be handled by the TCP sender is the arrival of an acknowledgment segment (ACK) from the receiver (more specifically, a segment containing a

valid ACK field value). On the occurrence of this event, TCP compares the ACK value y with its variable `SendBase`. The TCP state variable `SendBase` is the sequence number of the oldest unacknowledged byte. (Thus `SendBase-1` is the sequence number of the last byte that is known to have been received correctly and in order at the receiver.) As indicated earlier, TCP uses cumulative acknowledgments, so that y acknowledges the receipt of all bytes before byte number y . If $y > \text{SendBase}$, then the ACK is acknowledging one or more previously unacknowledged segments. Thus, the sender updates its `SendBase` variable; it also restarts the timer if there currently are any not-yet-acknowledged segments.

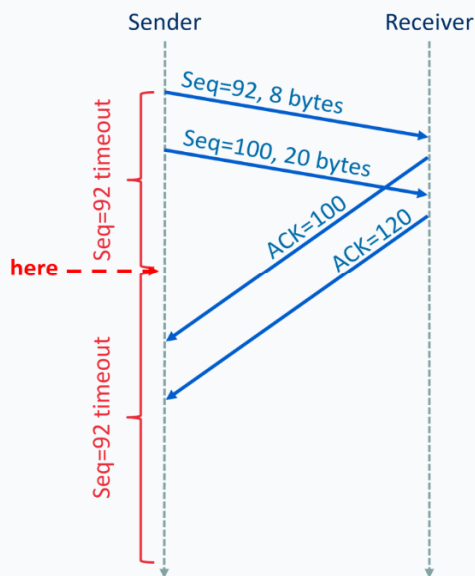
Exercise: Retransmission timeout in TCP

What happens after the first timeout of segment 92?

- A. Only segment 92 will be resent
- B. Segments 92 and 100 will be both be resent
- C. Segment 92 will be resent, and 100 will only be resent after the next timeout
- D. Neither segment will be resent



What happens after the first timeout of segment 92?



Only segment 92 will be resent

Segments 92 and 100 will both be resent

Segment 92 will be resent, and 100 will only be resent after the next timeout

Neither of the segments will be resent

Start the presentation to see live content. For screen share software, share the entire screen. Get help at pollev.com/app

Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.

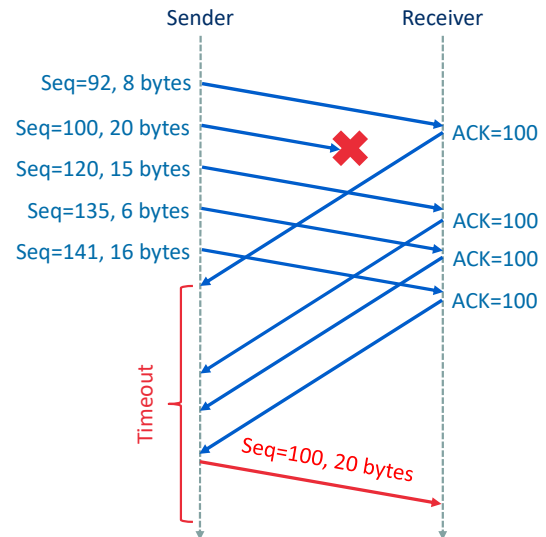
More info at polleverywhere.com/support

What happens after the first timeout of segment 92?

https://www.polleverywhere.com/multiple_choice_polls/bulsnlleSNbKmNAilZy1I?display_state=instructions&activity_state=opened&state=opened&flow=Engagement&onscreen=persist

Fast retransmission

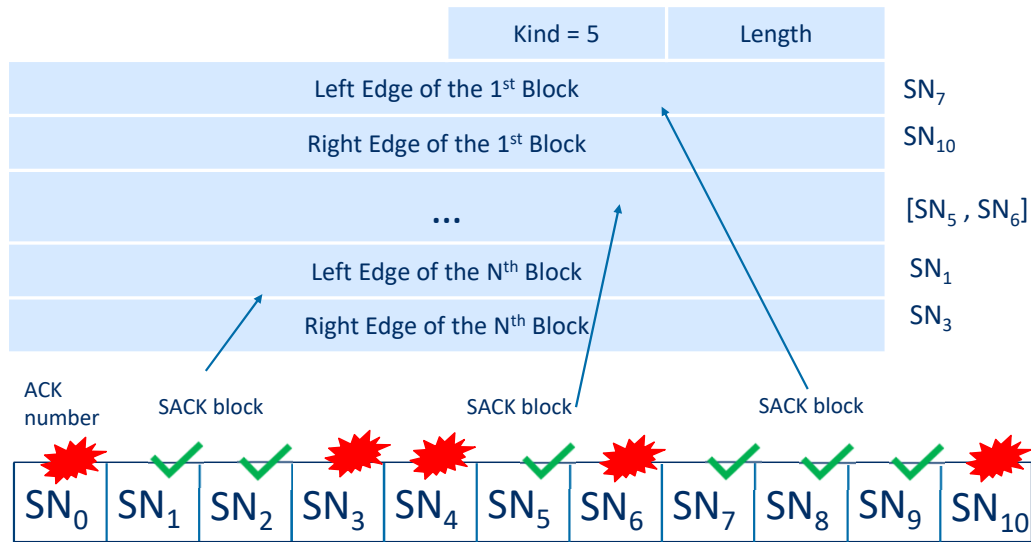
- The RTO value can be relatively long, especially for high-speed (e.g., Gigabit Ethernet) and high-RTT (e.g., satellites) networks.
- Lost packets can be detected earlier by counting **duplicate ACKs**, whose acknowledge number corresponds to already acknowledged segment.
- After **3 duplicate ACKs**, the oldest non-acknowledged segment is retransmitted.
- If a retransmission timeout occurs, it is an indication of significant problems or changes in the network.



One of the problems with timeout-triggered retransmissions is that the timeout period can be relatively long. When a segment is lost, this long timeout period forces the sender to delay resending the lost packet, thereby increasing the end-to-end delay. Fortunately, the sender can often detect packet loss well before the timeout event occurs by noting so-called duplicate ACKs. A **duplicate ACK** is an ACK that reacknowledges a segment for which the sender has already received an earlier acknowledgment.

Because a sender uses pipelining to transmit multiple segments, if one segment is lost, there will likely be multiple back-to-back duplicate ACKs. If the TCP sender receives three duplicate ACKs for the same data, it takes this as an indication that the segment following the segment that has been ACKed three times has been lost. In this case, the TCP sender performs a **fast retransmit** [RFC 5681], retransmitting the missing segment *before* that segment's timer expires.

TCP Selective Acknowledgement (SACK) option



TCP Selective Acknowledgement (SACK) option provides TCP sender with information about contiguous blocks of data successfully received by TCP receiver beyond the sequence number indicated in the Acknowledgement number field of TCP header. RFC 2018 defines the usage of this option. Using information from this SACK Option, TCP sender avoids retransmission of already successfully delivered segments.

Is TCP's error recovery Go-Back-N (GBN) or Selective Repeat (SR)?

Go-Back-N

Selective Repeat

A little of both

Neither

Start the presentation to see live content. For screen share software, share the entire screen. Get help at pollev.com/app

Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.

More info at polleverywhere.com/support

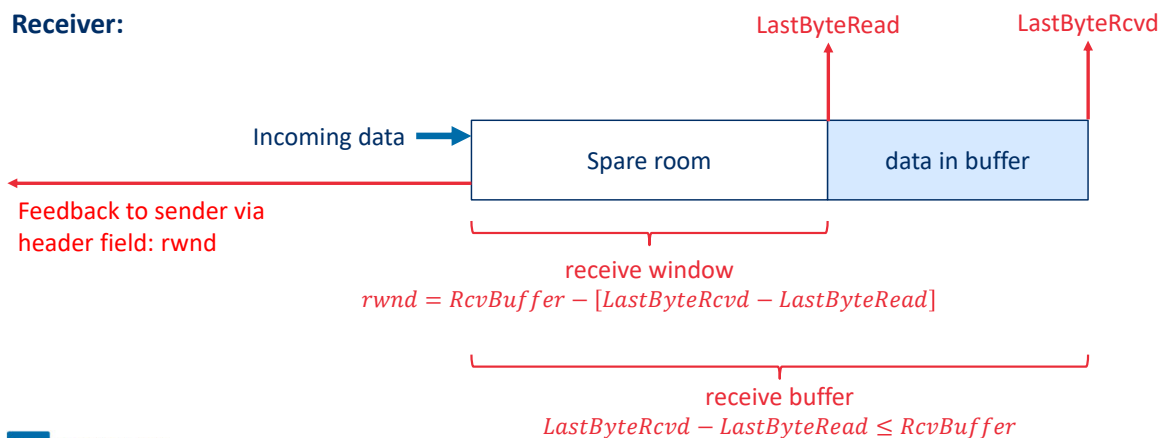
Is TCP's error recovery Go-Back-N (GBN) or Selective Repeat (SR)?

https://www.polleverywhere.com/multiple_choice_polls/oVw8JpGybgDwRv62MhqUG?display_state=instructions&activity_state=opened&state=opened&flow=Engagement&onscreen=persist

Flow control – Receiver

Ensures that the sender does not send data faster than the receiver application reads it

Receiver:



Recall that the hosts on each side of a TCP connection set aside a receive buffer for the connection. When the TCP connection receives bytes that are correct and in sequence, it places the data in the receive buffer. The associated application process will read data from this buffer, but not necessarily at the instant the data arrives. Indeed, the receiving application may be busy with some other task and may not even attempt to read the data until long after it has arrived. If the application is relatively slow at reading the data, the sender can very easily overflow the connection's receive buffer by sending too much data too quickly. TCP provides a **flow-control service** to its applications to eliminate the possibility of the sender overflowing the receiver's buffer. Flow control is thus a speedmatching service—matching the rate at which the sender is sending against the rate at which the receiving application is reading.

TCP provides flow control by having the *sender* maintain a variable called the **receive window**. Informally, the receive window is used to give the sender an idea of how much free buffer space is available at the receiver. Because TCP is full-duplex, the sender at each side of the connection maintains a distinct receive window. Let's investigate the receive window in the context of a file transfer. Suppose that Host A is sending a large file to Host B over a TCP connection. Host B allocates a receive buffer to this connection; denote its size by *RcvBuffer*. From time to time, the application process in Host B reads from the buffer. Define the following variables:

- *LastByteRead*: the number of the last byte in the data stream read from the buffer by the application process in B
- *LastByteRcvd*: the number of the last byte in the data stream that has arrived from the network and has been placed in the receive buffer at B

Because TCP is not permitted to overflow the allocated buffer, we must have:

$$LastByteRcvd - LastByteRead \leq RcvBuffer$$

The receive window, denoted *rwnd* is set to the amount of spare room in the buffer:

$$rwnd = RcvBuffer - [LastByteRcvd - LastByteRead]$$

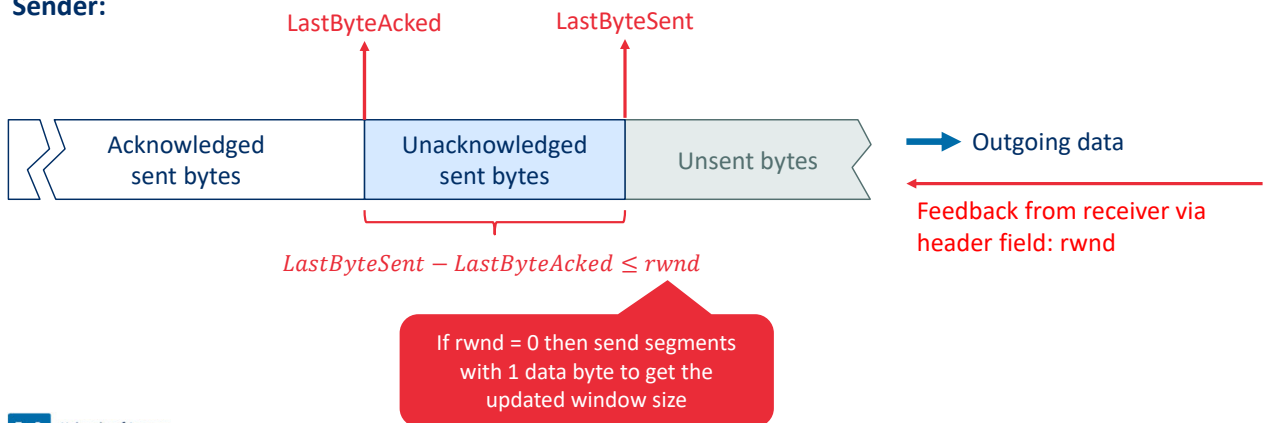
Because the spare room changes with time, *rwnd* is dynamic.

How does the connection use the variable *rwnd* to provide the flow-control service? Host B tells Host A how much spare room it has in the connection buffer by placing its current value of *rwnd* in the receive window field of every segment it sends to A. Initially, Host B sets *rwnd* = *RcvBuffer*. Note that to pull this off, Host B must keep track of several connection-specific variables.

Flow control – Sender

Ensures that the sender does not send data faster than the receiver application reads it

Sender:



Host A in turn keeps track of two variables, *LastByteSent* and *LastByteAked*, which have obvious meanings. Note that the difference between these two variables, $LastByteSent - LastByteAked$, is the amount of unacknowledged data that A has sent into the connection. By keeping the amount of unacknowledged data less than the value of *rwnd*, Host A is assured that it is not overflowing the receive buffer at Host B. Thus, Host A makes sure throughout the connection's life that

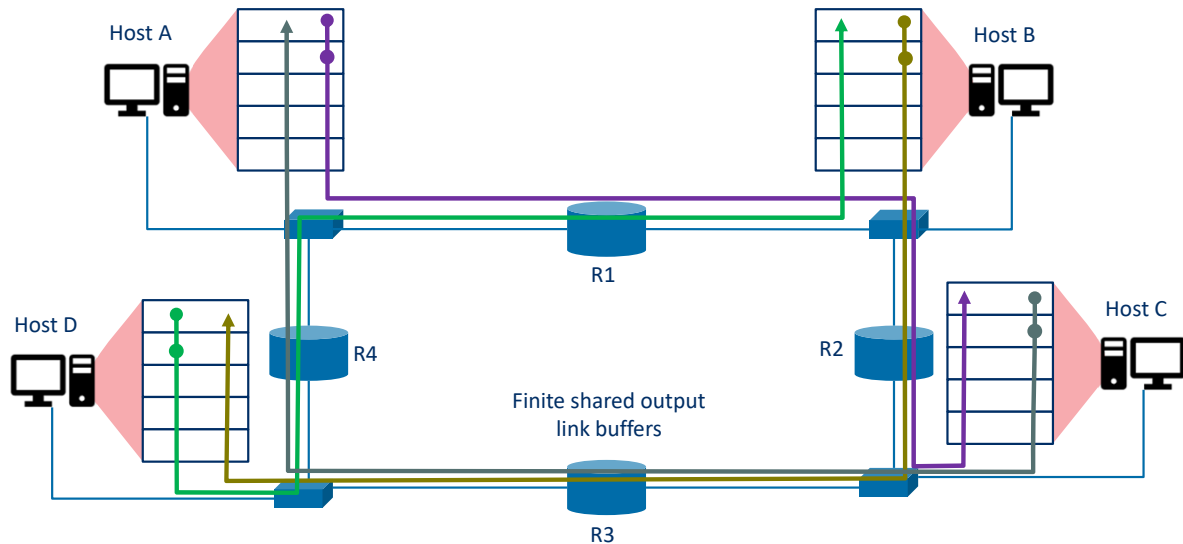
$$LastByteSent - LastByteAked \leq rwnd$$

There is one minor technical problem with this scheme. To see this, suppose Host B's receive buffer becomes full so that $rwnd = 0$. After advertising $rwnd = 0$ to Host A, also suppose that B has *nothing* to send to A. Now consider what happens. As the application process at B empties the buffer, TCP does not send new segments with new *rwnd* values to Host A; indeed, TCP sends a segment to Host A only if it has data to send or if it has an acknowledgment to send. Therefore, Host A is never informed that some space has opened up in Host B's receive buffer— Host A is blocked and can transmit no more data! To solve this problem, the TCP specification defines zero-window probing [RFCs 1122,9293], which requires Host A to continue to send segments with one data byte, when B's receive window is zero. These segments will be acknowledged by the receiver, and the updated receive window size will be indicated. Eventually the buffer will begin to empty and the acknowledgments will contain a non-zero *rwnd* value. According to RFC 1122, the transmitting host SHOULD send the first zero-window probe when a zero window has existed for the retransmission timeout period, and SHOULD increase exponentially the interval between successive probes.

TCP congestion control

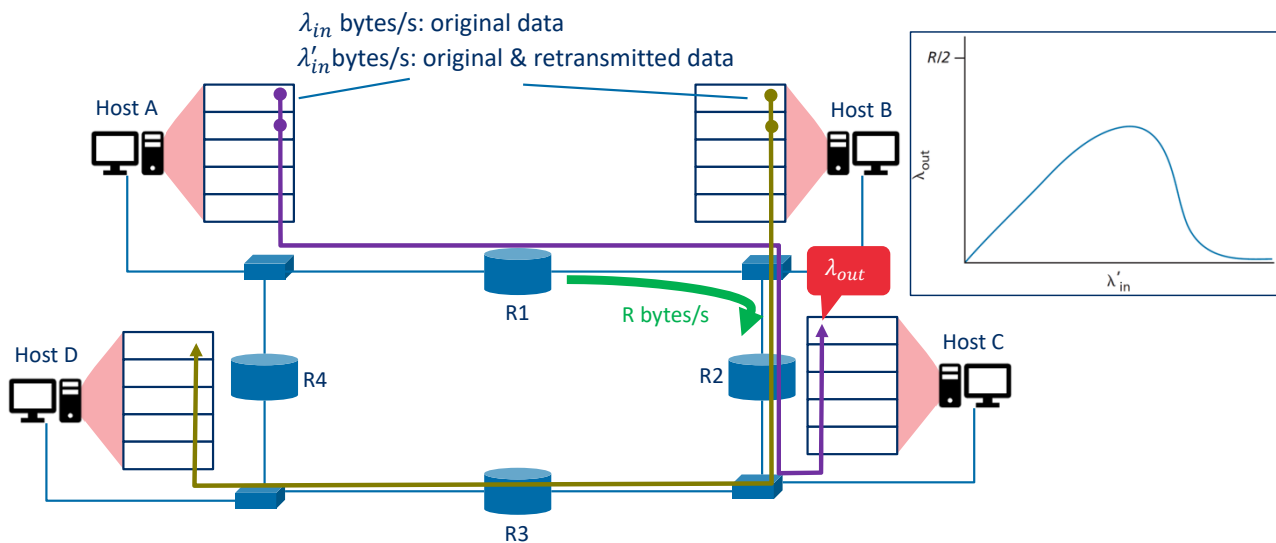
Packet retransmission treats a symptom of network congestion (the loss of a specific transport-layer segment) but does not treat the cause of network congestion—too high load produced by many sources sending data at too high rate. To treat the cause of network congestion, mechanisms are needed to throttle senders in the face of network congestion. We will first consider the problem of congestion control in a general context, seeking to understand why congestion is a bad thing, how network congestion is manifested in the performance received by upper-layer applications, and various approaches that can be taken to avoid, or react to, network congestion. Subsequently, we perform a detailed study of TCP's congestion-control algorithm.

Understanding congestion



In our congestion scenario, four hosts transmit packets, each over overlapping two-hop paths. We assume that each host uses a timeout/retransmission mechanism to implement a reliable data transfer service, that all hosts have the same value of λ_{in} , and that all router links have capacity R bytes/sec. The rate at which the transport layer sends segments (containing original data *and* retransmitted data) into the network will be denoted λ'_{in} bytes/sec. λ'_{in} is sometimes referred to as the **offered load** to the network.

Understanding congestion



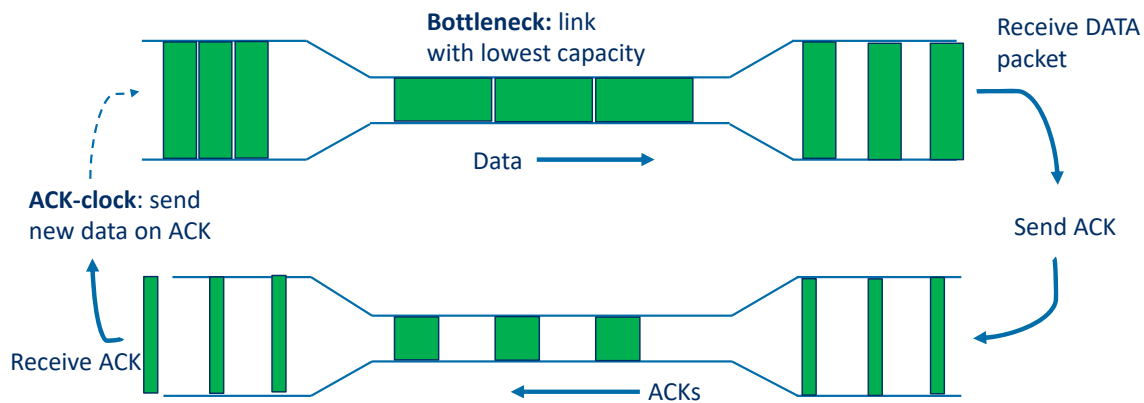
Let's consider the connection from Host A to Host C, passing through routers R1 and R2. The A–C connection shares router R2 with the B–D connection. For extremely small values of λ_{in} , buffer overflows are rare, and the throughput approximately equals the offered load. For slightly larger values of λ_{in} , the corresponding throughput is also larger, since more original data is being transmitted into the network and delivered to the destination, and overflows are still rare. Thus, for small values of λ_{in} , an increase in λ_{in} results in an increase in λ_{out} .

Having considered the case of extremely low traffic, let's next examine the case that λ_{in} (and hence λ'_{in}) is extremely large. Consider router R2. The A–C traffic arriving to router R2 (which arrives at R2 after being forwarded from R1) can have an arrival rate at R2 that is at most R , the capacity of the link from R1 to R2, regardless of the value of λ_{in} . If λ'_{in} is extremely large for all connections (including the B–D connection), then the arrival rate of B–D traffic at R2 can be much larger than that of the A–C traffic. Because the A–C and B–D traffic must compete at router R2 for the limited amount of buffer space, the amount of A–C traffic that successfully gets through R2 (that is, is not lost due to buffer overflow) becomes smaller and smaller as the offered load from B–D gets larger and larger. In the limit, as the offered load approaches infinity, an empty buffer at R2 is immediately filled by a B–D packet, and the throughput of the A–C connection at R2 goes to zero. This, in turn, *implies that the A–C end-to-end throughput goes to zero* in the limit of heavy traffic.

The reason for the eventual decrease in throughput with increasing offered load is evident when one considers the amount of wasted work done by the network. In the high-traffic scenario outlined above, whenever a packet is dropped at a second-hop router, the work

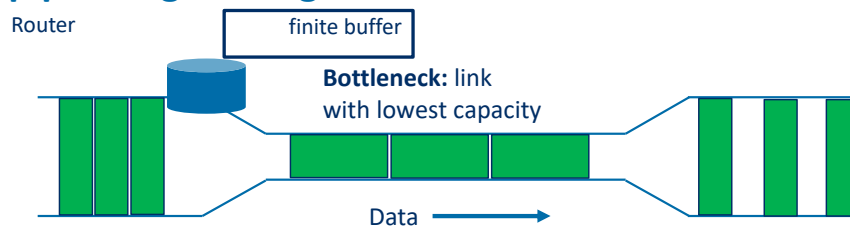
done by the first-hop router in forwarding a packet to the second-hop router ends up being “wasted.” The network would have been equally well off (more accurately, equally bad off) if the first router had simply discarded that packet and remained idle. More to the point, the transmission capacity used at the first router to forward the packet to the second router could have been much more profitably used to transmit a different packet. (For example, when selecting a packet for transmission, it might be better for a router to give priority to packets that have already traversed some number of upstream routers.) *So here we see yet another cost of dropping a packet due to congestion—when a packet is dropped along a path, the transmission capacity that was used at each of the upstream links to forward that packet to the point at which it is dropped ends up having been wasted.*

Understanding congestion: bottleneck



When a new TCP connection starts out, it does not know how many packets it should pipeline to reach optimal performance. The operation of TCP is driven or “clocked” by the receipt of ACKs. Receipt of an ACK indicates that one or more packets have been removed from the network, and consequently that an opportunity to send more has arisen. Since not all links along the path of the packet have the same capacity, there is always a bottleneck link, which has the lowest available capacity. On the picture, this link is shown as a narrow pipe, where packets “stretched out” in time. The capacity of this link slows determines how fast the data packets can be transmitted to reach a “steady-state” situation, where the ACK arrives just in time to trigger the transmission of a next packet, i.e., when the network is able to accept another packet. This relationship is sometimes called self-clocking.

Optimal pipelining for single data flow



Optimal amount of data “in flight”, i.e., sent but not acknowledged, is determined by:

- Bottleneck link capacity C bytes/s;
- Round-trip time RTT s.

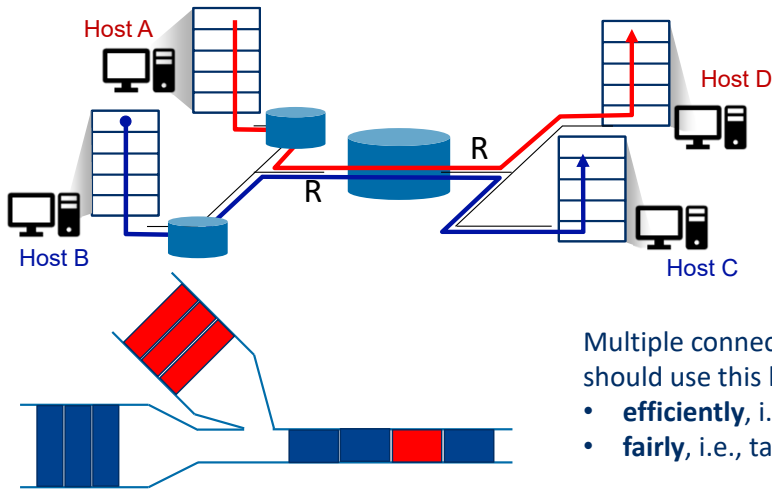
To fully fill the bottleneck link with packets “back-to-back” **without queueing**, the sender should pipeline $C \cdot RTT$ bytes. This number is called **bandwidth-delay product (BDP)**.

When the sender starts transmitting more than BDP data, the queueing will start happening. When the router’s buffer will become full, this router will start dropping packets.*

*In practice, it will start dropping packets earlier to avoid bursty drops.

When operating efficiently, there is no significant extra space between the packets in the bottleneck. The optimal number of packets “in flight”, i.e., that are sent but not acknowledged, should be around bandwidth-delay product (BDP) value, which is equal to $C \cdot RTT$ bytes, where C is the capacity of the bottleneck, and RTT is the round-trip time. This number of packets can “fill the pipe” without need for packet queueing. However, in practice it is not easy to determine the value of BDP. Hence, the algorithms used by TCP try to increase the window until the queueing comes into play, and the packets start dropping.

Fairness for multiple data flows



When multiple connections share the same bottleneck, they should use the available capacity efficiently and fairly. The efficient use means that the link utilization should be close to its full capacity R , and fairness means that TCP connections should share the link equally.

Summary: goals of congestion control

- Throttle (i.e., decrease transmission rate) senders in the face of network congestion
- Efficiently utilize the capacity of the bottleneck to maximize throughput
- Fairly share the bottleneck link between multiple connections

Two main approaches to congestion control

End-to-end congestion control

- Presence of congestion is inferred by end systems based on observed network behaviour
- Approach used by TCP
 - TCP packet loss is used to identify congestion*
 - TCP decreases window size accordingly

*In practice, packet loss can be caused not only by network congestion, but due to different other reasons. Notably, in wireless networks (such as Wi-Fi) many packet losses happen due to high variability of the wireless medium and interference.

Network-assisted congestion control

- Routers provide explicit feedback to the sender and/or receiver regarding congestion state
- Can be a single bit or more detailed feedback
- Revisited in relatively new standard L4S (RFC 9330) actively pushed by Nokia.
- Used by IBM SNA, DECnet, ATM, ...
- Supported by TCP, but many “middleboxes” (routers, firewalls, etc.) do not support it.

End-to-end congestion control. In an end-to-end approach to congestion control, the network layer provides no explicit support to the transport layer for congestion-control purposes. Even the presence of network congestion must be inferred by the end systems based only on observed network behavior (for example, packet loss and delay). TCP takes this end-to-end approach toward congestion control, since the IP layer is not required to provide feedback to hosts regarding network congestion. TCP segment loss (as indicated by a timeout or the receipt of three duplicate acknowledgments) is taken as an indication of network congestion, and TCP decreases its window size accordingly. Note that segment loss is not always caused by network congestion, and there are some mechanisms in TCP to distinguish between real congestion signals and spurious (e.g., RFC 5682).

Network-assisted congestion control. With network-assisted congestion control, routers provide explicit feedback to the sender and/or receiver regarding the congestion state of the network. This feedback may be as simple as a single bit indicating congestion at a link—an approach taken in the early IBM SNA, DEC DECnet architectures, and ATM network architectures. More sophisticated feedback is also possible. For example, in ATM Available Bit Rate (ABR) congestion control, a router informs the sender of the maximum host sending rate it (the router) can support on an outgoing link. More recently, L4S standard [RFC 9330] defined a modified version of TCP ECN to ensure low queuing latency, low congestion loss, and scalable throughput control. This standard is currently actively pushed by Nokia (<https://www.nokia.com/bell-labs/research/l4s/>).

TCP congestion control

- TCP limits the rate at which it sends traffic as a function of perceived network congestion

- This approach raises several questions

- How does a TCP sender limit its sending rate?

Sender uses congestion window (cwnd)
 $LastByteSent - LastByteAcked \leq \min(cwnd, rwnd)$

- How does a TCP sender perceive that there is congestion?

Timeout (severe problem) or three duplicate ACKs (congestion just started)

- What algorithm should the sender use to adapt its send rate as a function of congestion?

- Consists of 3 parts

- Slow start
 - Congestion avoidance
 - Fast retransmit & fast recovery

Decrease cwnd when segments are lost. Decrease stronger in case of timeout.
Increase cwnd when ACKs are received

The approach taken by TCP is to have each sender limit the rate at which it sends traffic into its connection as a function of perceived network congestion. If a TCP sender perceives that there is no congestion on the path between itself and the destination, then the TCP sender increases its send rate; if the sender perceives that there is congestion along the path, then the sender reduces its send rate. But this approach raises three questions. First, how does a TCP sender limit the rate at which it sends traffic into its connection? Second, how does a TCP sender perceive that there is congestion on the path between itself and the destination? And third, what algorithm should the sender use to change its send rate as a function of perceived end-to-end congestion?

Let's first examine how a TCP sender limits the rate at which it sends traffic into its connection. We saw that each side of a TCP connection consists of a receive buffer, a send buffer, and several variables (`LastByteRead`, `rwnd`, and so on). The TCP congestion-control mechanism operating at the sender keeps track of an additional variable, the **congestion window**. The congestion window, denoted `cwnd`, imposes a constraint on the rate at which a TCP sender can send traffic into the network. Specifically, the amount of unacknowledged data at a sender may not exceed the minimum of `cwnd` and `rwnd`, that is:

$$LastByteSent - LastByteAcked \leq \min(cwnd, rwnd)$$

The constraint above limits the amount of unacknowledged data at the sender and therefore indirectly limits the sender's send rate. To see this, consider a connection for which loss and packet transmission delays are negligible. Then, roughly, at the beginning of every RTT, the constraint permits the sender to send `cwnd` bytes of data into the connection; at the end of the RTT the sender receives acknowledgments for the data. Thus the sender's send rate is

roughly $cwnd/RTT$ bytes/sec. By adjusting the value of $cwnd$, the sender can therefore adjust the rate at which it sends data into its connection. Optimally, the send rate should correspond to the available bandwidth of the bottleneck link C , so that the optimal $cwnd$ value equals to the bandwidth-delay product $C \cdot RTT$ bytes. In practice, bandwidth-delay product is not easy to measure directly.

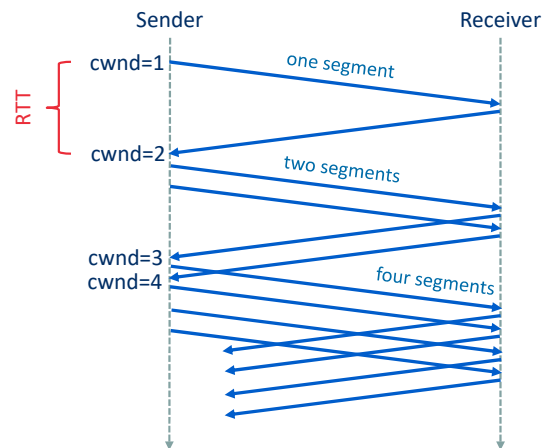
Let's next consider how a TCP sender perceives that there is congestion on the path between itself and the destination. Let us define a "loss event" at a TCP sender as the occurrence of either a timeout or the receipt of three duplicate ACKs from the receiver. When there is excessive congestion, then one (or more) router buffers along the path overflows, causing a datagram (containing a TCP segment) to be dropped. The dropped datagram, in turn, results in a loss event at the sender—either a timeout or the receipt of three duplicate ACKs—which is taken by the sender to be an indication of congestion on the sender-to-receiver path. A retransmission timeout is regarded as a more severe issue in the network, so the response to a timeout is stronger than to three duplicate ACKs.

Having considered how congestion is detected, let's next consider the more optimistic case when the network is congestion-free, that is, when a loss event doesn't occur. In this case, acknowledgments for previously unacknowledged segments will be received at the TCP sender. As we'll see, TCP will take the arrival of these acknowledgments as an indication that all is well—that segments being transmitted into the network are being successfully delivered to the destination—and will use acknowledgments to increase its congestion window size (and hence its transmission rate).

Given this overview of TCP congestion control, we're now in a position to consider the details of the celebrated **TCP congestion-control algorithm**, which is standardized in [RFC 5681]. The algorithm has three major components: (1) slow start, (2) congestion avoidance, and (3) fast recovery. Slow start and congestion avoidance differ in how they increase the size of $cwnd$ in response to received ACKs. We'll see shortly that slow start increases the size of $cwnd$ more rapidly (despite its name!) than congestion avoidance but starts from a very small $cwnd$ value. Fast recovery is used by TCP senders after fast retransmit is performed until the successful delivery of lost segment.

State 1: Slow start

- *cwnd* is initialized to *init_cwnd* MSS and increased by 1 MSS on every acknowledged MSS
 - “vanilla” slow start: *init_cwnd* = 1 MSS
 - RFC 5681: *init_cwnd* ~ 4000 bytes (3 MSS for MSS = 1460 bytes)
 - RFC 6928 (Linux default): *init_cwnd* = 10 MSS
- When *cwnd* ≥ *ssthresh* (slow start threshold), TCP transitions into **congestion avoidance mode**
- Initial value of *ssthresh* is Infinity (= no threshold)



When a TCP connection begins, the value of *cwnd* is typically initialized to a small value *init_cwnd* (typically ~10 MSS [RFC 6928]), resulting in an initial sending rate of roughly *init_cwnd*/RTT. For example, if MSS = 500 bytes and RTT = 200 msec, the resulting initial sending rate is only about 200 kbps. Since the available bandwidth to the TCP sender may be much larger than that, the TCP sender would like to find the amount of available bandwidth quickly. Thus, in the **slow-start** state, the value of *cwnd* begins at *init_cwnd* MSS and increases by 1 MSS every time a new MSS of data is acknowledged.

In the example, TCP sends the first segment into the network and waits for an acknowledgment. When this acknowledgment arrives, the TCP sender increases the congestion window by one MSS and sends out two maximum-sized segments. These segments are then acknowledged, with the sender increasing the congestion window by 1 MSS for each of the acknowledged segments, giving a congestion window of 4 MSS, and so on. This process results in a doubling of the sending rate every RTT. Thus, the TCP send rate starts slow but grows exponentially during the slow start phase.

But when should this exponential growth end? First, slow start stops when the value of *cwnd* reaches the value of a special variable, Slow Start Threshold (*ssthresh*). After that, a congestion avoidance phase starts. The initial value of variable *ssthresh* is set to infinity. This means that new TCP connection will always start from Slow Start and will keep exponentially increasing the window until the congestion is detected, i.e., until the packet is lost.

Given a slow start congestion control mechanism (default Linux implementation) with MSS = 500 bytes, RTT of 200 ms, and a link capacity of 10 Mbps, what is the initial sending rate?

500 bps

2.5 kbps

25 kbps

200 kbps

10 Mbps

Start the presentation to see live content. For screen share software, share the entire screen. Get help at pollev.com/app

Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.

More info at polleverywhere.com/support

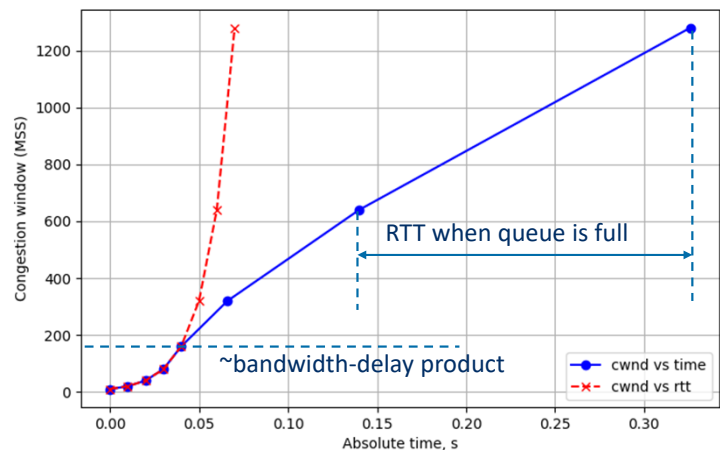
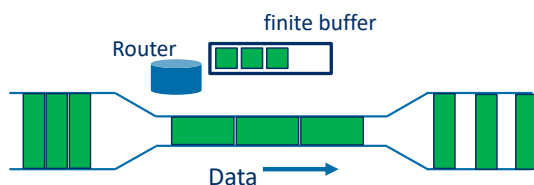
Given a slow start congestion control mechanism (default Linux implementation) with MSS = 500 bytes, RTT of 200 ms, and a link capacity of 10 Mbps, what is the initial sending rate?

https://www.polleverywhere.com/multiple_choice_polls/dWyzIs7RE51uYdOQqI3PK?display_state=instructions&activity_state=opened&state=opened&flow=Engagement&onscreen=persist

State 1: Slow start

cwnd is increased twice in every RTT.

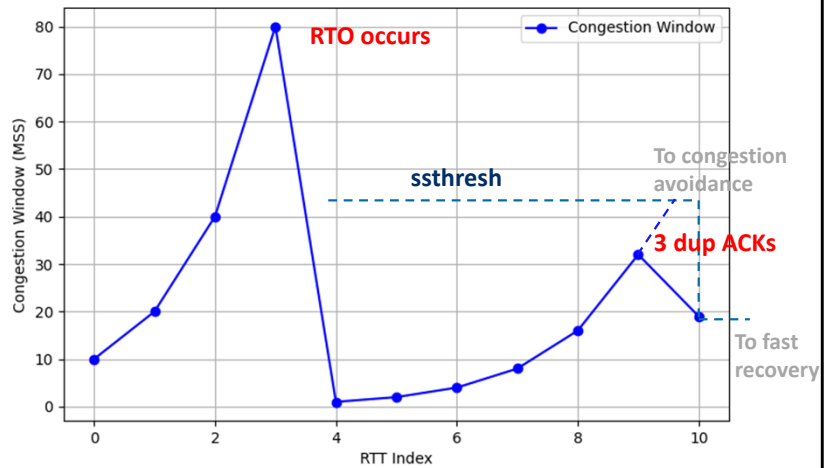
However, in absolute time scale, RTT starts increasing when *cwnd* reaches bandwidth-delay product.



Note that in slow start *cwnd* is increased twice every RTT. When we plot *cwnd* as a function of RTT index, we get a curve growing exponentially (red curve on the Figure). However, the packet will be dropped only when the router's queue will start growing. As a result, the RTT value will not be a constant value anymore. More specifically, RTT will grow with the number of packets in the queue. We already know that optimal *cwnd* is around bandwidth-delay product (BDP), because this is the maximum value that keeps the queues empty. When *cwnd* exceeds BDP, queueing at the router will introduce additional delay. The more is the difference between *cwnd* and BDP – the higher is the additional queueing delay.

State 1: Slow start

- When a **timeout** occurs
 - ssthresh is set to $\text{cwnd} / 2$
 - cwnd is reset to 1 MSS
- When 3 duplicate ACKs are received TCP transitions into **fast recovery** state
 - ssthresh is set to $\text{cwnd} / 2$
 - cwnd is set to $(\text{cwnd} / 2 + 3 \text{ MSS})$



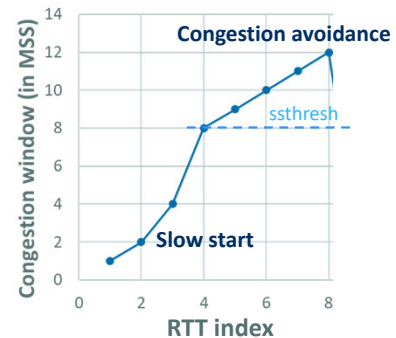
As we already know, there are two ways of packet loss detection.

First, if there is a loss event indicated by a timeout, the TCP sender sets the value of cwnd to 1 and begins the slow start process anew. It also sets the value of a second state variable, ssthresh, to $\text{cwnd}/2$ —half of the value of the congestion window value when congestion was detected.

Second, if three duplicate ACKs have been received, TCP performs fast retransmit and enters the fast recovery state. In this case, ssthresh is also set to $\text{cwnd}/2$, but the congestion window is not reset to 1 MSS. Instead, it is set to $\text{cwnd}/2 + 3 \text{ MSS}$. This way, TCP sender's transmission rate is reduced, but it doesn't need to start from scratch.

State 2: Congestion avoidance [TCP Reno, Additive Increase Multiplicative Decrease (AIMD)]

- Starts when $cwnd$ reaches $ssthresh$ (half of $cwnd$ value when congestion was last encountered)
- TCP only increases $cwnd$ by 1 MSS every RTT (instead of doubling it as in slow start mode)
 - Common approach: When new ACK arrives, $cwnd = cwnd + 1/cwnd$. The idea is to increase $cwnd$ by 1 after $cwnd$ ACKs are received (i.e., after ~ 1 RTT).
 - If $cwnd$ is measured in bytes, the formula transforms into $cwnd_bytes = cwnd_bytes + MSS^2 / cwnd_bytes$
- TCP goes to:
 - slow start state when a timeout occurs ($ssthresh = \frac{cwnd}{2}$; $cwnd = 1$);
 - fast recovery after 3 duplicate ACKs ($ssthresh = \frac{cwnd}{2}$; $cwnd = ssthresh + 3 \text{ MSS}$).



On entry to the congestion-avoidance state, the value of $cwnd$ is approximately half its value when congestion was last encountered—congestion could be just around the corner! Thus, rather than doubling the value of $cwnd$ every RTT, TCP adopts a more conservative approach and increases the value of $cwnd$ more slowly. The way this increase happens depends on the version of TCP used. There have been proposed many TCP congestion avoidance algorithms. We will consider two most famous, TCP Reno and TCP CUBIC. The latter is used by default in many modern operating systems.

In congestion avoidance state, TCP Reno increases $cwnd$ by just a single MSS every RTT [RFC 5681]. A common approach is for the TCP sender to increase $cwnd$ by $(MSS * MSS / cwnd)$ bytes whenever a new acknowledgment arrives. For example, if MSS is 1,460 bytes and $cwnd$ is 14,600 bytes, then 10 segments are being sent within an RTT. Each arriving ACK (assuming one ACK per segment) increases the congestion window size by $1/10$ MSS, and thus, the value of the congestion window will have increased by one MSS after ACKs when all 10 segments have been received.

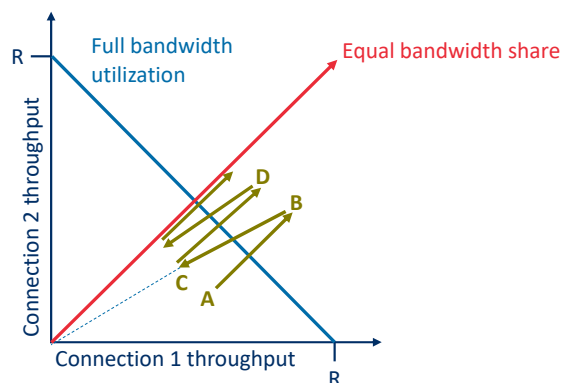
But when should congestion avoidance's linear increase (of 1 MSS per RTT) end? TCP's congestion-avoidance algorithm behaves the same when a timeout occurs as in the case of slow start: The value of $cwnd$ is set to 1 MSS, and the value of $ssthresh$ is updated to half the value of $cwnd$ when the loss event occurred. Recall, however, that a loss event also can be triggered by a triple duplicate ACK event. In this case, the network is continuing to deliver some segments from sender to receiver (as indicated by the receipt of duplicate ACKs). So, TCP's behavior to this type of loss event should be less drastic than with a timeout-indicated loss: TCP halves the value of $cwnd$ (adding in 3 MSS for good measure to account for the

triple duplicate ACKs received) and records the value of ssthresh to be half the value of cwnd when the triple duplicate ACKs were received. The fast-recovery state is then entered.

Ignoring the initial slow-start period when a connection begins and assuming that losses are indicated by triple duplicate ACKs rather than timeouts, TCP Reno's congestion control consists of linear (additive) increase in cwnd of 1 MSS per RTT and then a halving (multiplicative decrease) of cwnd on a triple duplicate-ACK event. For this reason, TCP congestion control is often referred to as an **additive-increase, multiplicative-decrease (AIMD)** form of congestion control.

Fairness of TCP Reno's AIMD approach

- A congestion control mechanism is **fair** if K connections sharing a bottleneck link with transmission rate R bps, each achieve an average transmission rate of approximately R/K .
- The TCP Reno's AIMD algorithm has been shown to be fair in certain cases.
- Let's consider an example of a link with transmission rate R shared by 2 TCP connections **with equal RTTs**



Consider K TCP connections, each with a different end-to-end path, but all passing through a bottleneck link with transmission rate R bps. (By *bottleneck link*, we mean that for each connection, all the other links along the connection's path are not congested and have abundant transmission capacity as compared with the transmission capacity of the bottleneck link.) Suppose each connection is transferring a large file and there is no UDP traffic passing through the bottleneck link. A congestion control mechanism is said to be *fair* if the average transmission rate of each connection is approximately R/K ; that is, each connection gets an equal share of the link bandwidth. Is TCP's AIMD algorithm fair, particularly given that different TCP connections may start at different times and thus may have different window sizes at a given point in time?

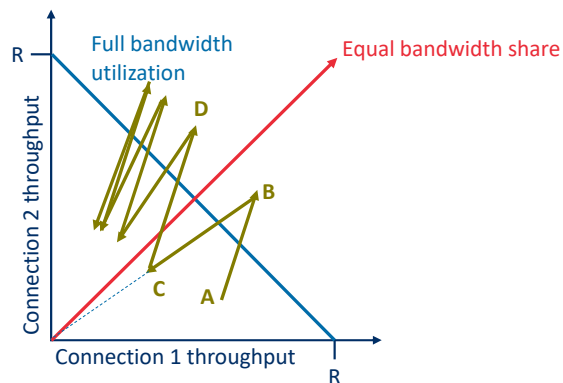
Let's consider the simple case of two TCP connections sharing a single link with transmission rate R . Assume that the two connections have the same MSS and RTT (so that if they have the same congestion window size, then they have the same throughput), that they have a large amount of data to send, and that no other TCP connections or UDP datagrams traverse this shared link. Also, ignore the slow-start phase of TCP and assume the TCP connections are operating in congestion avoidance mode (AIMD) at all times.

If TCP is to share the link bandwidth equally between the two connections, then the realized throughput should fall along the 45-degree arrow (equal bandwidth share) emanating from the origin. Ideally, the sum of the two throughputs should equal R . (Certainly, each connection receiving an equal, but zero, share of the link capacity is not a desirable situation!) So the goal should be to have the achieved throughputs fall somewhere near the intersection of the equal bandwidth share line and the full bandwidth utilization line.

Suppose that the TCP window sizes are such that at a given point in time, connections 1 and 2 realize throughputs indicated by point *A*. Because the amount of link bandwidth jointly consumed by the two connections is less than R , no loss will occur, and both connections will increase their window by 1 MSS per RTT as a result of TCP's congestion-avoidance algorithm. Thus, the joint throughput of the two connections proceeds along a 45-degree line (equal increase for both connections) starting from point *A*. Eventually, the link bandwidth jointly consumed by the two connections will be greater than R , and eventually packet loss will occur. Suppose that connections 1 and 2 experience packet loss when they realize throughputs indicated by point *B*. Connections 1 and 2 then decrease their windows by a factor of two. The resulting throughputs realized are thus at point *C*, halfway along a vector starting at *B* and ending at the origin. Because the joint bandwidth use is less than R at point *C*, the two connections again increase their throughputs along a 45-degree line starting from *C*. Eventually, loss will again occur, for example, at point *D*, and the two connections again decrease their window sizes by a factor of two, and so on. You should convince yourself that the bandwidth realized by the two connections eventually fluctuates along the equal bandwidth share line. You should also convince yourself that the two connections will converge to this behavior regardless of where they are in the two-dimensional space! Although a number of idealized assumptions lie behind this scenario, it still provides an intuitive feel for why TCP results in an equal sharing of bandwidth among connections.

RTT unfairness of TCP Reno's AIMD approach

- A congestion control mechanism is **fair** if K connections sharing a bottleneck link with transmission rate R bps, each achieve an average transmission rate of approximately R/K .
- The TCP Reno's AIMD algorithm relies on ACK clocks. The connection with smaller RTT will increase CWND faster. Besides, throughput formula (approx. cwnd/RTT) has RTT in denominator.
- Let's consider an example of a link with transmission rate R shared by 2 TCP connections **with non-equal RTTs** ($\text{RTT}_1 > \text{RTT}_2$).



The realized throughput during additive increase does not fall anymore along the 45-degree arrow. Indeed, when $\text{RTT}_1 > \text{RTT}_2$, the second connection increases its window faster, so the arrow is tilted more towards the second connection. After multiple iterations, we can see that the convergence point favors the second connection more.

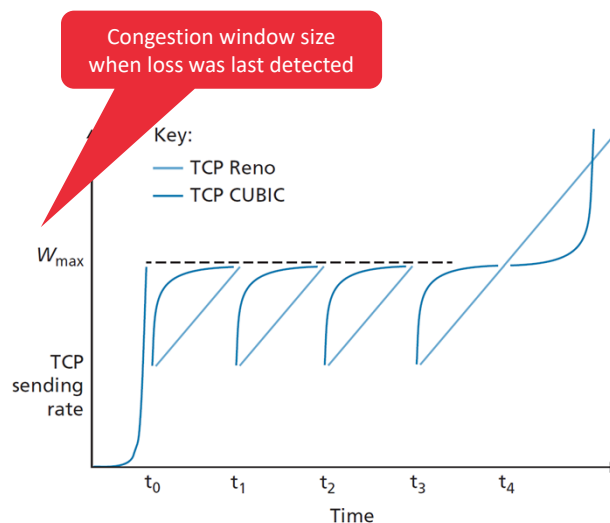
State 2: Congestion avoidance [TCP CUBIC, default in many OS]

TCP Reno

- Additive-increase, multiplicative-decrease (AIMD)
- Slowly increases cwnd after packet loss
- Window increases by $\frac{MSS^2}{cwnd}$ bytes on every ACK

TCP CUBIC

- Quickly ramps up sending rate to get close to pre-loss sending rate
- Probes cautiously afterwards
- Window increase function depends on the time elapsed since congestion event, not on ACKs arrival rate => better RTT-fairness.



Given TCP Reno's additive-increase, multiplicative-decrease approach to congestion control, one might naturally wonder whether this is the best way to "probe" for a packet sending rate that is just below the threshold of triggering packet loss. Indeed, cutting the sending rate in half (or even worse, cutting the sending rate to one packet per RTT as in an earlier version of TCP known as TCP Tahoe) and then increasing rather slowly over time may be overly cautious. If the state of the congested link where packet loss occurred hasn't changed much, then perhaps it's better to more quickly ramp up the sending rate to get close to the pre-loss sending rate and only then probe cautiously for bandwidth. This insight lies at the heart of a flavor of TCP known as TCP CUBIC.

TCP CUBIC differs from TCP Reno only in congestion avoidance, while slow start and fast recovery are the same. Once again, in congestion avoidance, the congestion window is increased only on ACK receipt, but the increase value depends on the time elapsed since the congestion event. Specifically, CUBIC changes the congestion avoidance phase, as follows:

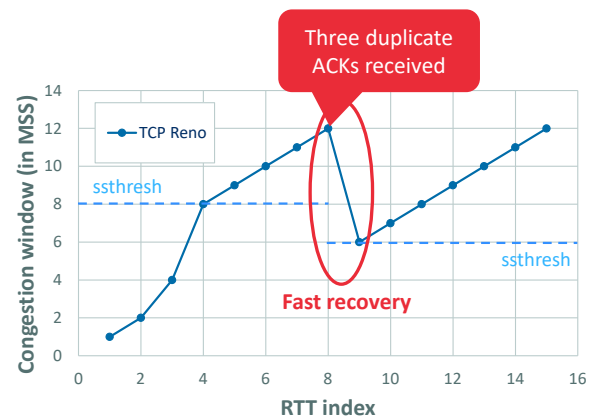
- Let W_{max} be size of TCP's congestion control window when loss was last detected, and let K be the future point in time when TCP CUBIC's window size will again reach W_{max} , assuming no losses. Several tunable CUBIC parameters determine the value K , that is, how quickly the protocol's congestion window size would reach W_{max} .
- CUBIC increases the congestion window as a function of *cube* of the distance between the current time, t , and K . Thus, when t is further away from K , the congestion window size increases are much larger than when t is close to K . That is, CUBIC quickly ramps up TCP's sending rate to get close to the pre-loss rate, W_{max} , and only then probes cautiously for bandwidth as it approaches W_{max} .
- When t is greater than K , the cubic rule implies that CUBIC's congestion window

increases are small when t is still close to K (which is good if the congestion level of the link causing loss hasn't changed much) but then increases rapidly as t exceeds K (which allows CUBIC to more quickly find a new operating point if the congestion level of the link that caused loss has changed significantly).

The graph compares TCP Reno and CUBIC. We see the slow start phase ending at t_0 . Then, when congestion loss occurs at t_1 , t_2 , and t_3 , CUBIC more quickly ramps up close to $W_{\max}$ (thereby enjoying more overall throughput than TCP Reno). We can see graphically how TCP CUBIC attempts to maintain the flow for as long as possible just below the (unknown to the sender) congestion threshold. Note that at t_3 , the congestion level has presumably decreased appreciably, allowing both TCP Reno and TCP CUBIC to achieve sending rates higher than $W_{\max}$.

State 3: Fast recovery

- Fast recovery starts when 3 duplicate ACKs are received during slow start or congestion avoidance
 - $ssthresh$ is set to $cwnd / 2$
 - $cwnd = ssthresh + 3 \cdot MSS$
- On each next duplicate ACK $cwnd$ is increased by 1 MSS (window inflation)
- When a **timeout** occurs, TCP goes to slow start (setting $cwnd$ to 1 MSS and $ssthresh$ to $cwnd / 2$)
- When an **ACK arrives** for the missing segment, TCP goes back to congestion avoidance. $cwnd$ is reset to $ssthresh$ (window deflation)



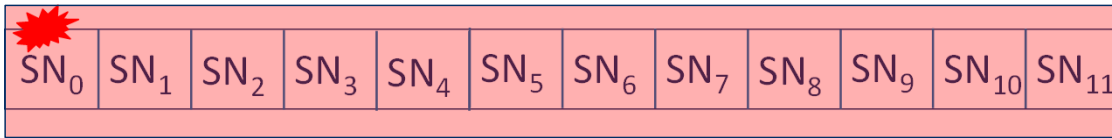
In fast recovery, the value of $cwnd$ is increased by 1 MSS for every duplicate ACK received for the missing segment that caused TCP to enter the fast-recovery state ($cwnd$ inflation to keep ACK clock and not waste resources). Eventually, when an ACK arrives for the missing segment, TCP enters the congestion-avoidance state after deflating $cwnd$. If a timeout event occurs, fast recovery transitions to the slow-start state after performing the same actions as in slow start and congestion avoidance: The value of $cwnd$ is set to 1 MSS, and the value of $ssthresh$ is set to half the value of $cwnd$ when the loss event occurred.

It is interesting that an early version of TCP, known as TCP Tahoe, unconditionally cut its congestion window to 1 MSS and entered the slow-start phase after either a timeout-indicated or triple-duplicate-ACK-indicated loss event. The newer versions of TCP incorporated fast recovery.

The graph illustrates the evolution of TCP's congestion window for TCP Reno. In this figure, the slow start threshold is initially equal to 8 MSS. For the first eight transmission rounds, the congestion window climbs exponentially fast during slow start and hits the threshold at the fourth round of transmission. The congestion window then climbs linearly until a triple duplicate-ACK event occurs, just after transmission round 8. Note that the congestion window is $12 \times MSS$ when this loss event occurs. The value of $ssthresh$ is then set to $0.5 \times cwnd = 6 \times MSS$. The congestion window is set to $cwnd = 9 \times MSS$, and then fast recovery starts. After recovery of the lost segment is finished, $cwnd$ is dropped back to $ssthresh$ value, i.e., $6 \times MSS$, and congestion avoidance starts. Note that for a single loss without further losses fast recovery will take just one RTT, that is why we don't see it on the figure.

State 3: Fast recovery

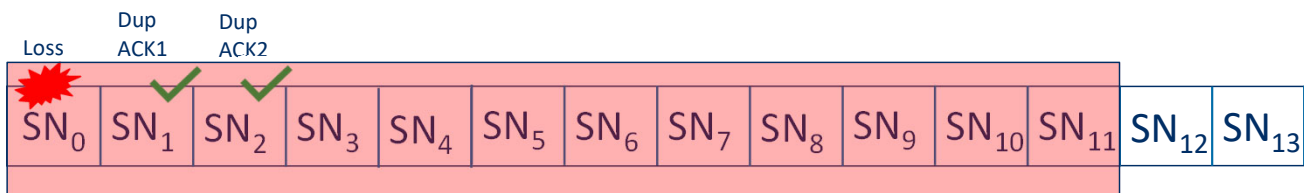
Loss



TCP segment with sequence number SN_0 is lost.

Current congestion window size $cwnd = 12 \text{ MSS}$

State 3: Fast recovery

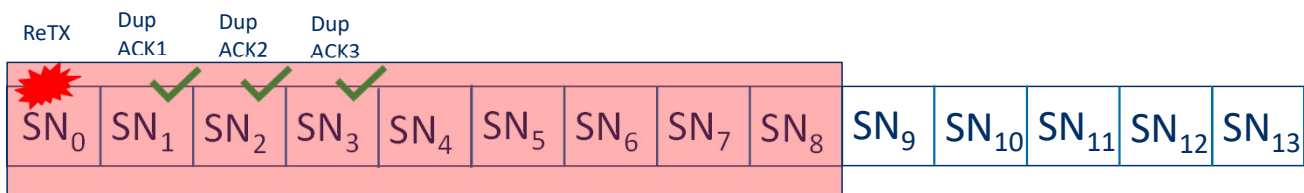


First two duplicate ACKs arrived.

ACKs confirm that some data segments have been received => two data segments has been removed from pipe.
TCP transmits 2 segments to substitute these segments (limited transmit) [RFC 3042].

Segments with sequence numbers SN₁₂ and SN₁₃ are transmitted.

State 3: Fast recovery

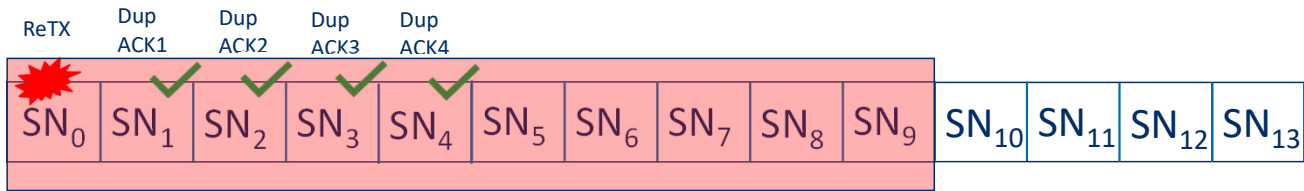


Third duplicate ACK arrived.

Fast retransmit is performed. TCP retransmits segment with sequence number SN_0 .

Fast recovery starts. Slow start threshold is set to $ssthresh = 6 \text{ MSS}$, congestion window is set to $cwnd = ssthresh + 3 \text{ MSS} = 9 \text{ MSS}$.

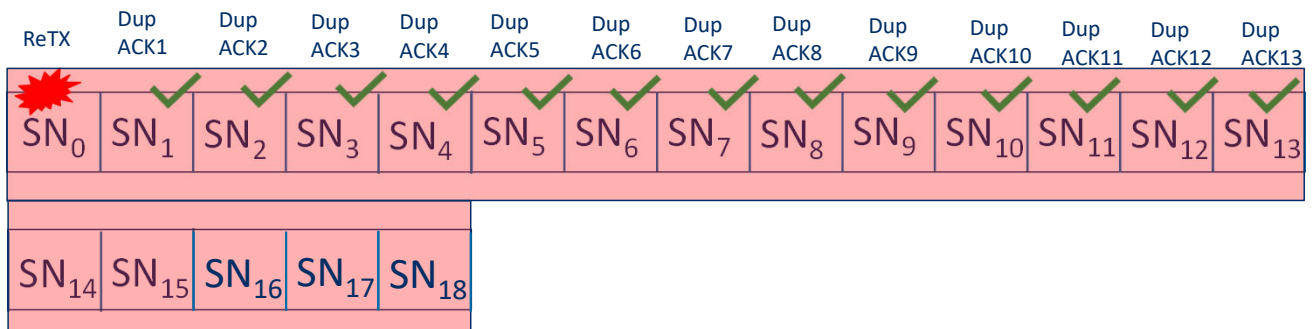
State 3: Fast recovery



Fourth duplicate ACK arrived.

One more segment left the pipe => TCP “inflates” window to compensate (TCP conservation law). Since segment with SN_9 has been already sent, nothing will be sent.

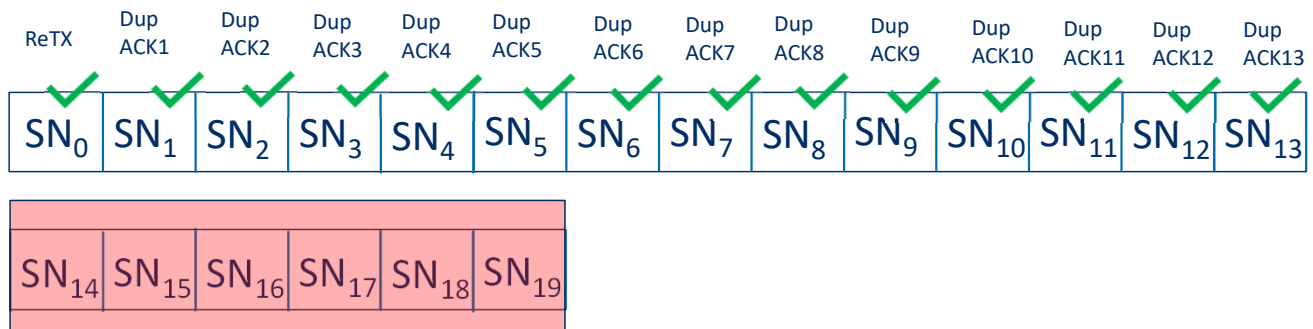
State 3: Fast recovery



13th duplicate ACK arrived.

TCP continue “inflating” window. New segments with sequence numbers SN₁₄, SN₁₅, SN₁₆, SN₁₇, and SN₁₈ are sent.

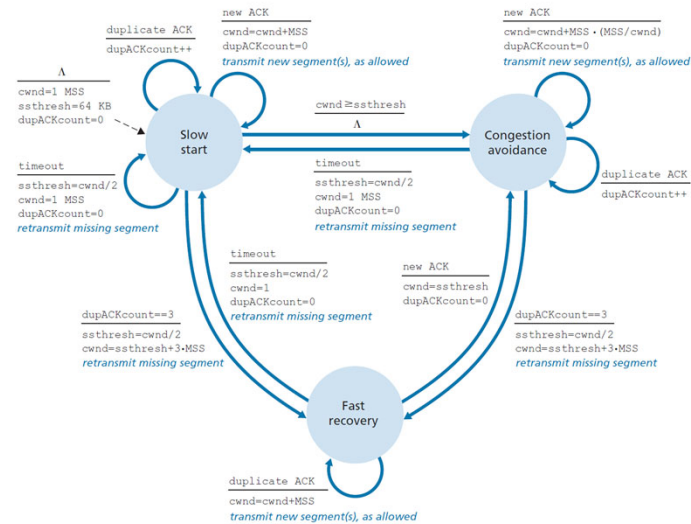
State 3: Fast recovery



Acknowledgement for TCP segment with sequence number SN₀ finally arrives. Fast recovery is finished.

TCP deflates congestion window. The new value is: $cwnd = ssthresh = 6 \text{ MSS}$. Segment with sequence number SN₁₉ is sent. TCP congestion avoidance start.

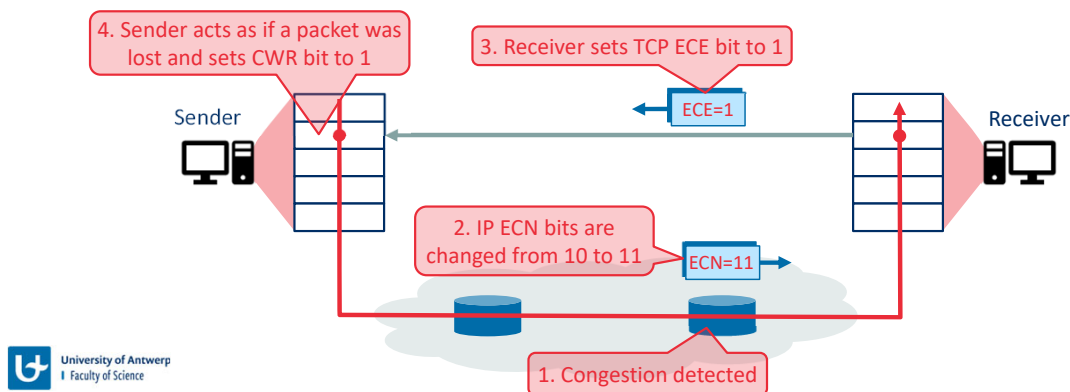
Summary: FSM description of TCP congestion control



The figure presents the complete FSM description of TCP's congestion-control algorithms (for TCP Reno case) — slow start, congestion avoidance, and fast recovery. The figure also indicates where transmission of new segments or retransmitted segments can occur. Although it is important to distinguish between TCP error control/retransmission and TCP congestion control, it's also important to appreciate how these two aspects of TCP are inextricably linked.

Explicit congestion notification (ECN) [RFC 3168]

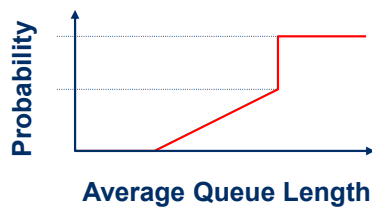
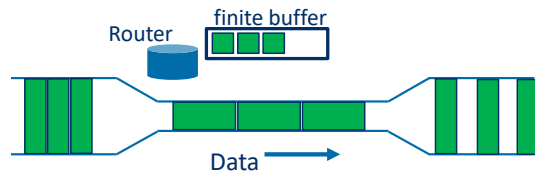
- Uses 2 bits in the **IP** header
 - ECN-capable transport (ECT) bit is set to 1 if the sender and receiver support ECN
 - Congestion Experienced (CE) bit is set to 1 if an intermediary router experiences congestion
- What qualifies as congestion is left to the equipment vendor and/or network operator to define
- If the CE bit is 1 on a received TCP packet, the receiver will set the TCP ECE bit to 1 in its ACK
- If the sender receives an ACK with ECE=1 then it acts as if a packet was lost and sets CWR=1 in next segment



Explicit Congestion Notification [RFC 3168] is the form of network-assisted congestion control performed within the Internet. Both TCP and IP are involved. At the network layer, two bits (with four possible values, overall) in the Type of Service field of the IP datagram header are used for ECN. One setting of the ECN bits is used by a router to indicate that it (the router) is experiencing congestion. This congestion indication is then carried in the marked IP datagram to the destination host, which then informs the sending host. RFC 3168 does not provide a definition of when a router is congested; that decision is a configuration choice made possible by the router vendor, and decided by the network operator. However, the intuition is that the congestion indication bit can be set to signal the onset of congestion to the send before loss actually occurs. A second setting of the ECN bits is used by the sending host to inform routers that the sender and receiver are ECN-capable, and thus capable of taking action in response to ECN-indicated network congestion.

When the TCP in the receiving host receives an ECN congestion indication via a received datagram, the TCP in the receiving host informs the TCP in the sending host of the congestion indication by setting the ECE (Explicit Congestion Notification Echo) bit in a receiver-to-sender TCP ACK segment. The TCP sender, in turn, reacts to an ACK with a congestion indication by reducing the congestion window, as it would react to a lost segment using fast retransmit, and sets the CWR (Congestion Window Reduced) bit in the header of the next transmitted TCP sender-to-receiver segment.

Active queue management



- When the buffer of the router becomes full, it is already too late to start dropping the packets.
 - long delay because of waiting full queue of packets;
 - butch-losses because multiple packets arrive to the fully occupied buffer
- Active queue management addresses this issue. The idea is to drop packets earlier to throttle TCP senders
- Example: *Random Early Detection (RED)* is designed to drop packets randomly. The probability of dropping increases with the queue length.

Summary

Summary

- The transport layer provides a logical end-to-end connection between processes
 - (De)multiplexing uses port numbers and IP addresses to associate packets with processes
 - Bit errors in received packets are detected using a checksum header field
- UDP is a “bare bones” transport protocol that does not provide any additional functionality
- TCP offers additional features, such as reliability, congestion control and flow control
 - Reliability is achieved by combining acknowledgments, timers, retransmissions and sequence numbers
 - TCP estimates the RTT to properly configure the retransmission timeout
 - Flow control allows the receiver to limit the transmission rate using the receive window header field
 - TCP adapts its congestion window when a timeout occurs or it receives three duplicate ACKs
- The contents of this part is covered in the **book** in Sections 3.1 – 3.7

End of Part 3